Jot Complete Specification

Version 1.0.0-draft.3 | 26 October 2026 | Proposed standard

A plain-text journal format, with a packaged form that carries the writing and its images, scans and handwriting in one file.

Changed in draft 3: the two representations have their own extensions. Plain-text Jot is `.jot`; the packaged document is `.jotz` (section 2.0).

The writing comes first

Jot takes its main inspiration from Fountain: a readable plain-text writing format that an application can interpret and typeset. A `.jot` file is that writing itself, as plain UTF-8 text. The primary `.jotz` document packages the same writing as ordered `.jot` entry members, together with any images, scans, handwriting or other assets, even when a journal is text-only. Either MAY contain a single moment, one day, a whole month, several years, a topic collection, or an ongoing journal. File boundaries MUST NOT imply date boundaries. The filename MUST NOT be treated as authoritative journal content. A date heading organizes writing inside a document; it never requires a separate daily file.

A writer does not need metadata, JSON, a form, a method declaration, a database, or a structured record. The following is complete journal writing, a whole `.jot` file (or the entry source inside a `.jotz`), combining several styles across two days:

```
# October 2026

### 2026-10-25 | Sunday

.0830 - KITCHEN

The rain made the room feel quieter. I wanted to remember that.

#### A small plan
[ ] Photograph the connector label @home
o Tea with a friend
- Enough for today: start with one small thing.

#### Gratitude
The kettle was already full. Someone remembered my favorite tea.

#### What I noticed
The shop was loud. Headphones helped more than pushing through.

### 2026-10-26 | Monday

#### Morning pages
I don't know what to write yet, so I'll start here...

= There was more room when I left the plan small.
```

The paragraphs are the journal, not descriptions of external journal entries. Ordinary headings such as "Gratitude" or "Morning pages" provide enough structure for a human. Their wording is never a required magic keyword, and software MUST NOT infer health states or task completion from them. Optional structured records later in this specification exist for reliable queries, measurements, links and interchange. They are an extension layer; they MUST NOT become the default authoring requirement or replace prose with a form.

The method examples below show optional structured representations alongside human writing prompts. An editor SHOULD offer a writing-first view and MAY expose structured properties separately. PDF pages, monthly collections and daily views are renderings or selections of the same writing. Splitting a month into daily files, or joining daily files into a month, is an explicit file operation that preserves text order and stable references.

Reading this specification

This document expands the original DotJot v1.0 PDF (`references/Historical-DotJot-Specification.pdf`) into a language specification, an interoperable data model, and a catalogue of method profiles. It covers every one of the 46 method/style guides present in the Journalwright methods library at snapshot `f67a54dd20c9` (copied into this repository as

`docs/methods/`), plus complementary profiles. The repository is the source of method coverage; the syntax, profile contracts, examples and implementation requirements below are new Jot design proposals.

This is a complete design within its declared scope, not a claim that every journaling tradition has been enumerated or that a parser fully implements every clause. Profiles adapt practices rather than reproducing commercial workbooks, certifying a clinical instrument, or claiming product compatibility. Medical and psychological profiles specify how to record information; they do not prescribe treatment or assert therapeutic effectiveness. The repository's research claims have not been independently audited for this specification.

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY express required, recommended and optional behavior. Normative sections define implementation behavior. Profile explanations and examples are informative unless explicitly stated otherwise. All personal content in examples is fictional.

Contents

1. Goals and conformance
2. Files, encoding and document structure
3. Deterministic parsing and grammar
4. Shared content primitives
5. Records, measurements and time
6. Tasks, collections and relationships
7. Method profiles and composition
8. Presentation and optional page layouts
9. Privacy, authorship and portability
10. Importing the original outline
11. AST and validation contract
12. Conformance fixtures and acceptance criteria
13. Method catalogue and worked examples
14. Implementation sequence and source register

1. Goals and conformance

The writing inside Jot is readable as ordinary text. A short diary paragraph is valid journal content; a `.jotz` package carries it with any assets. Structure is progressive: a writer can add dates, task marks, measurements, links or templates only when useful. A single entry MAY combine any number of profiles. The format MUST preserve unsorted thoughts, unfinished sentences, empty days, nonstandard labels and unknown extensions without inventing meaning.

Jot has two representations with the same writing syntax, each with its own extension:

Extension	Representation	Media type until registered
`.jot`	Plain Jot: the writing as UTF-8 text (section 2.1). Readable and editable in any text editor. Cannot embed binary assets.	`text/plain; charset=utf-8`
`.jotz`	Packaged Jot: a versioned ZIP archive with ordered entry source, UUID media, optional per-entry encryption and a public-only SQLite/FTS5 search index for archives with at least 256 KiB of public writing (section 2.0).	`application/zip`

Writer MUST default to `.jotz` for every journal, including text-only journals. Plain `.jot` and Markdown remain explicit source import/export. Read content by signature, then validate container version. Both forms belong to Jot v1; earlier archive prototypes and `.jotlock` have no migration layer. A tool that handles only text MAY list `.jot` and MUST NOT save ZIP bytes as text. Writing-only exports omit assets and protection and must say so.

Three capabilities are separately reportable. **Core Reader** parses structural blocks and preserves all source. **Profile Processor** additionally validates one or more named, versioned profiles. **Renderer** additionally produces a declared screen or print layout. A device supporting only Core MUST remain useful for free writing and capture; it MUST preserve unsupported profile blocks. Claiming Jot compatibility MUST list these capabilities, supported versions, limits and extensions. A file validator MUST distinguish syntax errors, profile issues, unresolved links and presentation warnings.

The format is local-first and independent of cloud services, AI, a particular editor or a specific notebook. Handwriting, scans, audio and art can remain primary sources. Text recognition is optional and MUST NOT silently replace the original.

2. Files, encoding and document structure

2.0 Self-contained document container (`.jotz`)

The default journal is Jotz container version 1.0. The text grammar remains Jot v1. Normative container and encryption details are in Jot-Container-v1.md and Jot-Protection-v1.md.

Offline completeness is a required v1 release condition. A complete `.jotz` must contain everything needed to read its saved content in a locally available compatible reader: entries, included media/ink/documents, integration snapshots and required presentation resources. Reading must not require provider accounts, expiring URLs, cloud services or AI. Protected entries need a local unlocking route. External citations remain optional links with readable

local labels; they do not require copying the internet. Required remote content must be bundled lawfully or clearly reported as incomplete. Reader runtime resources must also be locally available. See the container specification for acceptance criteria and current prototype gaps.

The first ZIP member is a bounded, uncompressed UTF-8 `manifest.json`. Ordered public entry members are exact UTF-8 source at `entries/<uuid>.jot`; protected entries, all attachments when protection is enabled, and document metadata use independently encrypted `private/<uuid>.enc` members. Public media is under `media/<uuid>`. `index.sqlite`, written once public entry source reaches 256 KiB, contains only public entries, tags and explicitly written measurements, with FTS5. Smaller archives omit it and search their source directly. It is a validated/rebuildable cache, never executable journal SQL or the source of truth. Private search uses unlocked source in memory.

Concatenating the ordered entry source reconstructs exact writing, including BOM, CRLF, repeated dates, preamble and unfinished prose. IDs are independent of dates. Logical journal date, declared timezone and actual known UTC creation/update instants remain separate; imported creation times may be unknown. Public entry source is readable after ZIP extraction; protected source is not. Dates, member counts/sizes/UUIDs and cryptographic slot metadata remain visible. A mixed archive is not fully encrypted.

Each protected member uses a separate random AES-256-GCM content key wrapped by an archive master key; passphrase/recovery/passkey slots open the master. Descriptor identity, revision and logical date are authenticated. An encrypted catalog binds the manifest's ordered members, index and key slots. Unchanged entry ciphertext may be reused, but ZIP saving can still rewrite a complete file. A privacy change builds a fresh public SQLite database so deleted private text cannot survive in FTS shadow/freelist pages. Existing copies/backups cannot be retroactively erased.

Validate all required members, safe paths, sizes, hashes and authentication before replacement. Reject unlisted files, duplicate names, traversal, absolute/backslash paths, symlinks, ZIP64, ZIP encryption and unsupported compression. Cache errors fall back to bounded source search. Prototype limits: 64 MiB input/expanded members, 32 MiB total source/per attachment, 1 MiB per entry, 4 MiB manifest, 16 MiB index, 10,000 entries, 1,000 assets and 12,000 ZIP members. These are resource bounds, not ten-year performance guarantees.

`parse(text)` still consumes ordinary source. `pack({source,entries,assets,secret,previous})` produces `.jotz` bytes; `readDocument(bytes,{secret})` returns complete source/assets after unlock, or only public content with `partial: true` without a secret. A partial document must not be rewritten as a complete journal. Plain import/export remains available, and the unmodified source plus exact asset bytes must round-trip. No migration layer is required for this fresh replacement.

2.1 Text and whitespace

Each entry source MUST be valid UTF-8; a leading UTF-8 BOM is accepted and retained in lossless mode. Readers accept LF and CRLF line endings and a missing final newline. Canonical writers emit LF and one final newline. Unicode content MUST be preserved without normalization; identifiers use ASCII. Source offsets count UTF-8 bytes from zero, with end exclusive. Lines and columns in diagnostics are one-based; columns count Unicode scalar values, with a tab counting as one column.

Outside literal blocks, tabs expand to the next multiple of four columns for indentation. Syntax markers are case-sensitive unless stated otherwise. Unknown text is prose, not an error. Blank lines separate paragraphs; consecutive ordinary lines in a paragraph form one paragraph with explicit source newlines retained. A renderer MAY visually reflow them. Two trailing spaces request a hard line break. Literal content never reflows. Where earlier Fountain-inspired rules conflict with ordinary Markdown formatting, the explicit Markdown compatibility rules in section 4.2 take priority in this draft.

2.2 Optional metadata

Metadata is recognized only between a first-line `---` and the next standalone `---`. It is deliberately not YAML. Each nonblank line is `Key: value`, split at the first colon. A key matches `[A-Za-z][A-Za-z0-9 -]*`. Values are trimmed strings; there are no escapes, implicit numbers, arrays or executable tags. Blank values are allowed. Keys are compared case-insensitively after trimming. Repeated keys remain ordered; singleton duplicates produce a warning and the last occurrence supplies the effective value. Unknown keys are preserved. An unclosed opening delimiter is ordinary prose with a warning.

```
---
Jot: 1.0
Document-ID: journal-main
Title: Everyday notebook
Author: writer-a
Timezone: America/Los_Angeles
Language: en
Theme: More room to breathe
Default Grid: dot-5mm
Privacy: private
---

### 2026-10-25 | Sunday

A line about today is enough.
```

`Jot` declares the major/minor language version. Missing version means Core 1.0 syntax in a newly created file, but an importer MUST ask for or explicitly select legacy interpretation for known original-outline sources. A reader MUST warn on an unsupported major version and offer read-only/raw preservation. `Document-ID`, when supplied, MUST match `[A-Za-z0-9][A-Za-z0-9._-]{0,127}`. Authors are local identifiers; display names MAY be stored separately. `Timezone` is an IANA identifier; unknown or missing zones remain unresolved, never silently replaced with the device zone. `Privacy` is `private`, `shared` or `public`, default `private`.

2.3 Headings, entries and periods

Headings start at column zero with one to six `#` characters and a space. Levels convey hierarchy, not fixed calendar meanings. A heading beginning with an exact ISO date (`YYYY-MM-DD`), optionally followed by `| label`, opens a dated entry and closes the previous entry regardless of level. Invalid dates remain headings with a diagnostic. Other headings remain within the current entry until the next dated heading. Content before the first dated heading belongs to an undated preamble entry. Duplicate dates form distinct entries in source order; no automatic merge occurs.

Year, month, week, season and collection headings are descriptive. Structured periods use a `period` record with `start` inclusive and `end` exclusive ISO dates. Week calculations use ISO Monday-based weeks when `calendar: iso-week` is declared. A daily view is a projection over entries, not a requirement to create a blank file every day. Leap days are valid; an anniversary view MUST omit an absent February 29 rather than silently move its entry.

`= ` begins a synopsis line. Multiple synopses are retained in order. `===` on its own is a page-break block, never an entry boundary. Heading text, dates and dates embedded in prose are never interpreted as scheduling commands.

2.3.1 Entry timezone override

A standalone `@timezone: Europe/London` directive inside a dated entry supplies its timezone, taking precedence over the document's `Timezone` metadata. It applies to the whole entry, regardless of where in it the directive occurs, and ends at the next dated heading, including a repeated date. Without an override, the document default applies. An undated-preamble override is a warning; use metadata for a document default. Zones use IANA names containing a region/path, or UTC/GMT; ambiguous bare abbreviations are not accepted. Invalid/blank declared zones remain unresolved and MUST NOT fall back to the device or to a previous valid value. Duplicates warn and use the last declaration. Source and written dates/times remain unchanged. Place timezone fields do not supply entry context. Code, comments and record content do not become timezone directives.

An exact RFC 3339 instant with offset retains its meaning. Calendar export may resolve a local dated event only when its effective zone yields exactly one instant; DST gaps and repeated hours require an explicit offset rather than a guessed occurrence. Date-only

entries, filters and all-day events retain their calendar dates. Separate entries can cover a day or a recap period; a `period` record describes coverage without changing entry-boundary or timezone scope.

3. Deterministic parsing and grammar

3.1 Block precedence

A parser MUST apply these rules in order: optional metadata at byte zero; active literal fence; active record block; new literal fence; new record opening; hidden comment; heading; page break; layout directive; forced scene; explicit voice; synopsis; rapid log; marginalia; centered line; paragraph. A block opener starts a new block even without a preceding blank line. Inline parsing happens only after block recognition. Journal extension markers are anchored to column zero except rapid-log continuation lines. Markdown blocks retain their own indentation rules.

```
document       = [metadata], {block} ;
metadata       = "---", NL, {metadata-line | blank}, "---", [NL] ;
heading        = "#"{1..6}, SP, text ;
synopsis       = "=", SP, text ;
pagebreak      = "===" ;
scene          = ".", nonspace, text ;
voice          = "::voice", SP, text ;
record-open    = ":::record", SP, name, [SP, "#", id] ;
record-close   = ":::" ;
directive      = "@", name, ":", [SP], text ;
task           = "[", state, "]", SP, text ;
state          = " " | "x" | ">" | "<" | "-" | "!" ;
log            = ("-" | "o" | "?" | "!"), SP, text ;
name           = lower, {lower | digit | "-"} ;
id             = alnum, {alnum | "." | "_" | "-"} ;
```

This compact grammar describes recognition, not the entire Unicode text grammar. `NL` is an accepted newline; `SP` is one ASCII space; `nonspace` excludes whitespace and newline. Record IDs use the same length and character restriction as document IDs. A fence is a line of at least three backticks, optionally followed by a language label. It closes at a column-zero line containing at least as many backticks and no other nonspace characters. Tilde fences are accepted under the Markdown rules, with the same marker and minimum closing length. Embedded syntax inside fences is always literal.

3.2 Literal escape and recovery

A backslash before the first marker at column zero suppresses block recognition for that line and displays the marker literally. Inline `\` escapes the next ASCII punctuation character; other backslashes remain text. A code span enclosed in single backticks suppresses inline interpretation; a backtick that has no partner is ordinary text. To quote longer text containing backticks, use a fenced block with a longer delimiter.

Record blocks cannot nest. An opening record encountered inside a record is invalid record content and remains raw. A missing closing `:::` consumes through EOF and produces an error with the raw bytes intact. An unclosed literal fence behaves similarly. Hidden comments start only with standalone `/*` and close at standalone `*/`; they cannot nest. An unclosed hidden comment consumes through EOF with a warning and MUST show a visible editor diagnostic. All invalid constructs MUST be recoverable as raw content and MUST NOT cause data deletion. Unknown directives and record types are valid opaque blocks.

3.3 Fountain-inspired constructs

`.0830 - WORKSHOP` is a forced scene label. The Core Reader stores its whole label and MUST NOT infer an actual timestamp from `0830`. `::voice ME` starts an explicit voice cue; subsequent paragraphs belong to that cue until a heading, scene, voice cue or record boundary. A standalone parenthetical immediately after a voice cue is an optional delivery/context note. Ordinary ALL-CAPS lines and words such as `LOG:`, `ENTRY:`, `INT.` and `EXT.` remain prose in Core; heuristic recognition exists only in the legacy importer. This prevents diary text such as "I AM TIRED" from becoming an unintended character cue.

4. Shared content primitives

4.1 Rapid capture and lists

`[ ]` means open task; `[x]` completed; `[>]` migrated; `[<]` scheduled; `[-]` cancelled. `[!]` is a v1-compatible open task with priority `high`; it is not a separate lifecycle state. `- ` is a Markdown list item that may serve as a rapid note; `o ` is an event, `? ` a question, and `! ` an idea. Markers require one following space; bare `[x]` is prose. Two-space-indented nonblank lines continue the preceding rapid-log item, stripped of exactly two spaces for display. Other indents are preserved as text. Nested actionable tasks SHOULD use records and `parent` links; indentation alone MUST NOT create hidden task relationships.

Inline contexts use `@name` and tags use `#name` at a word boundary, with names containing Unicode letters/digits, `_` or `-`. They end at whitespace or punctuation. Jot does not identify email addresses as contexts because `@` must be at the start or preceded by whitespace. `#` alone never starts a tag. Inside a heading, heading recognition wins; inline tags in its text

remain possible. Leading `* ` is an ordinary prose list marker; use `[!]` or a task record for machine-readable priority. A renderer MAY make simple prose lists without assigning task semantics.

4.2 Formatting and annotations

Markdown is the base writing syntax. Implementations SHOULD use an established CommonMark/GFM parser and declare its version. Support headings, paragraphs, nested ordered/unordered lists, links, images as attachment references, emphasis, blockquotes, fenced/indented code, tables, task lists and strikethrough. `*italic*` and `_italic_` follow Markdown; `**bold**` is bold; `~~text~~` is strikethrough. The original outline's underline and single-tilde lyric conventions require explicit legacy import and MUST NOT override Markdown by default. Raw HTML is retained as source but is not executed; renderers MUST disable or sanitize it.

`- [ ] task` and `- [x] task` are Markdown task-list items; bare `[ ] task` and `[x] task` are Jot rapid task markers. Ordinary blockquotes beginning `> ` follow Markdown. The old `> centered <` convention is legacy-only; a renderer may offer explicit alignment properties. Standard fenced and indented code suppresses journal commands. Setext heading underlines take priority when `===` immediately follows writing; a standalone `===` after a blank line is a Jot page break. Unknown fenced languages remain literal source.

`[[annotation text]]` is marginalia, not a wiki link. It is single-line and ends at the next unescaped `]]`. Links use `[label](target)` with nonempty label and a target containing no unescaped whitespace or parentheses. Internal targets use `#id` or `other.jot#id`; external targets use explicit `https:` or `http:` schemes. Raw web addresses remain readable text. No script execution is permitted. Nested markup within annotations or link labels remains literal. Overlapping delimiters are consumed left-to-right with precedence: escape, code, link, marginalia, emphasis, tags/contexts. No construct starts inside code.

4.2.1 Fuller formatting

Jot adds six constructs that common Markdown editors share. Each is presentation only: none creates a record, a task or a link target.

Source	Meaning
`==text==`	highlighted (rendered as `mark`)
`++text++`	underlined (rendered as `ins`)
`^text^`	superscript; `text` contains no whitespace, `^` or brackets
`[^label]`	a footnote reference; `label` is 1 to 40 of `A-Z a-z 0-9 _ -`
`[^label]: text` at the start of a line	the footnote; indented (two spaces or a tab) lines continue it

Source	Meaning
`-> text <-` alone on a line	a centred line
`> [!NOTE]` as the whole first line of a blockquote	a callout: `NOTE`, `TIP`, `IMPORTANT`, `WARNING` or `CAUTION`, case-insensitive, as GitHub alerts

An opening `==` or `++` must be followed, and its closing mark preceded, by a non-whitespace character; the closing mark is not followed by the same character; the content does not cross a blank line. So `a == b`, `C++` and `2 ^ 3` remain text. They are inline and nest with emphasis, in the precedence order above (after code and links, before emphasis). Footnotes are numbered by first reference; a renderer gathers them at the end of the page or document, and a footnote never referenced is still shown. A reference to a missing footnote stays a visible reference. `~text~` remains text (no subscript).

Inline colors use `[text]{ink=#rrggbb highlight=#rrggbb strike=#rrggbb underline=#rrggbb}`. One or more attributes may appear, separated by a single space; each key occurs at most once. Color values are exactly six hexadecimal digits with a leading `#` (case-insensitive). `ink` changes the text color; `highlight` adds a background; `strike` and `underline` add independently colored decorations. Omitted attributes inherit the surrounding presentation. `highlight-style=marker`, `highlight-style=brush` or `highlight-style=pencil` optionally selects the handwritten highlight stroke (default: marker). Handwritten strokes vary by passage and a saved rendering seed; the same seed must reproduce the same preview and print, and an explicit Rewrite may change it without changing the writing. Renderers without handwriting may use a solid background. Spans may nest and contain inline Markdown and balanced/escaped brackets, but cannot cross a source newline. Code remains literal. Unknown keys, invalid values and duplicate keys do not form a color span. Renderers must validate the allowlist even for externally supplied ASTs and must never interpret these values as arbitrary CSS or HTML. Plain-text exports retain the readable syntax. Handwritten preview and print use the same colors; printing hardware and browser color settings may affect physical results.

A Markdown blockquote preserves quoted writing; a `quotation` record additionally stores explicit attribution and source fields. Hidden comments are omitted from rendered journal pages but retained by default in editable exports. They are neither encrypted nor safe places for secrets.

4.2.2 Links within and across documents

A link is `[label](#target)`. Two things in a document are link targets, and nothing else is:

- a record's ID: `:::record note #plan` is the target `#plan`;
- a dated heading's date: `### 2026-10-25 | Sunday` is the target `#2026-10-25`. When several headings share a date, the first is the target; a link to that date resolves to it and carries an `ambiguous-reference` warning, while repeated dates with no link to them remain valid

(§2.3). A record ID equal to a date takes precedence over the heading.

Links are found wherever inline writing is: paragraphs, headings, list items, table cells, blockquotes, footnotes, tasks, events, questions, ideas, synopses, voice lines and margin notes. Code and escaped brackets are never links. Record fields `relation: kind #target`, `member`, `source`, `parent`, `migrated-to` and `prompt` are links when their value (for `relation`, the part after the first space) is exactly `#target` or `path.jot#target`; any other value, such as a book title in `source`, is prose.

A `#target` naming nothing in the document, or naming a duplicated record ID (§4.3), is an `unresolved-reference` warning at the link. The link stays visible as written and never prevents writing. `other.jot#id` (or `.jotz`, a path relative to the current document) names another document: a reader keeps it as written, does not open, fetch or validate that file, and does not flag it. Links do not include, embed or copy content.

A renderer gives each target an element ID unique to that rendering (for example `jotln-<page>-2026-10-25`, so two previews in one page never clash) and points resolved links at it. It MAY list beside a target where it is linked from: the dated entry, or the record, that holds each link. Unresolved links render as text, not as a dangling anchor.

4.2.3 Spotify short references

`spotify:<type>:<id>` is an inline reference to Spotify, optionally with spaces or tabs after `spotify:`. Types are lowercase `track`, `album`, `playlist`, `artist`, `episode` and `show`; IDs are exactly 22 case-sensitive ASCII letters/digits. For example, `spotify: album:4aawyAB9vmqN3uQ7FjRGTy` expands in a capable reader to `https://open.spotify.com/album/4aawyAB9vmqN3uQ7FjRGTy`, labelled “Spotify album”. A Markdown link may supply a name: `[My writing mix](spotify:playlist:37i9dQZF1DXcBWIGoYBM5M)`. Several references can appear in an entry, including in lists. Readers MUST preserve source and MUST NOT fetch metadata, embed a player or start playback merely because a reference exists. Code spans, code blocks and escaped references stay literal. Unknown types or malformed bare references stay ordinary text; unsupported URI targets are displayed as text rather than opened. These references do not package audio or confer playback rights.

4.2.4 IMDb and YouTube short references

`imdb:tt0133093` identifies an IMDb title (film, series or episode); the ID is `tt` plus 7–10 decimal digits. It expands to `https://www.imdb.com/title/tt0133093/`. `youtube:dQw4w9WgXcQ`, optionally `youtube:video:dQw4w9WgXcQ`, identifies an 11-character video ID and expands to `https://www.youtube.com/watch?v=dQw4w9WgXcQ`. `youtube:playlist:<id>` expands to `https://www.youtube.com/playlist?list=<id>`; playlist IDs here contain 10–100 characters. YouTube IDs use case-sensitive ASCII letters/digits, underscore and hyphen. Horizontal space after the provider colon is optional. Provider names are lowercase.

Bare references have “IMDb title”, “YouTube video” or “YouTube playlist” labels; named Markdown links supply a writer's label, for example `[Background film](imdb:tt0133093)`. As in §4.2.3, preserve source, code and escapes, retain malformed/unknown bare forms as text, and perform no metadata, thumbnail or playback request during parsing/rendering. IMDb links identify works, not a streaming source. A shortened video reference does not encode timestamps; keep a full URL when timestamps matter.

4.2.5 MusicBrainz identity references

`musicbrainz:<entity>:<UUID>` identifies music independently of a playback provider. Entity types are `recording`, `release-group`, `release`, `work` and `artist`; the UUID has 8-4-4-4-12 hexadecimal groups, case-insensitive. Expand it to `https://musicbrainz.org/<entity>/<lowercase-UUID>`. Horizontal space after the provider colon is optional. Bare references show a “MusicBrainz ” label; Markdown links may supply a name. Preserve source and perform no lookup when parsing or rendering. A recording is not a composition, and an album release group is not an edition; readers must not silently substitute these or infer a matching Spotify/YouTube recording. Optional search must disclose the provider and let the writer select a match.

4.2.6 Document filename variable

`{{document.filename}}` is an optional inline presentation variable. In rendered prose, headings, lists and other inline text, it displays the current document's filename, including its extension. The host supplies this context; the parser MUST NOT infer it from metadata, dates or journal content, query a filesystem, or store the resolved value in place of the source token. Only the final filename component MAY be displayed, never parent directories or a full path. Display control characters as spaces. The name of a `.jotz` document is its outer filename, not any internal entry member.

```
# Notes from {{document.filename}}
```

With filename context `October.jot`, this heading displays “Notes from October.jot”. Renaming and rendering with the new context updates the display; saving the source still retains `{{document.filename}}`. This does not make filenames authoritative journal content (§2): names MUST NOT supply entry dates, stable identifiers, measurements or other journal facts.

The token is exact and case-sensitive, without interior whitespace. Backslash-escaping its first opening brace (`\{{document.filename}}`), inline code and code blocks keep it literal. Metadata, record field values, IDs, link/image targets, URLs and executable syntax are never rewritten. A renderer MAY resolve the token in the inline presentation of a textual record

caption without changing its field value or meaning. Insert the resolved name as plain text, never as Markdown or HTML, and do not evaluate a variable embedded inside the filename itself. This is not a general expression or template language.

Unknown variables and a known variable without available filename context MUST remain visible in their original spelling and MUST NOT trigger a lookup or error. For a new unsaved document a host MAY supply its proposed filename; it SHOULD distinguish this from a chosen file. A host rendering an included document MUST use that included file's basename within its frame, including its footnotes, and restore the enclosing document's context afterwards. A formatted export captures the displayed name at export time; a plain `.jot` download keeps the variable. Renderers without variable support can display the original text unchanged.

4.2.7 Entry scratchpads

A fenced literal block whose info string is `draft` is the **scratchpad** of the entry that contains it: a rough draft kept beside the finished writing.

```
### 2026-11-04 | Wednesday

~~~draft
bus late again, rain, saw the heron by the canal?? write about the heron
~~~

The heron was back by the canal this morning, standing in the rain as if it had been there
all night.
```

Its content is literal (3.2): tasks, tags, links, records and formatting inside it mean nothing, so rough notes never become tasks or measurements. A renderer MUST omit scratchpads from formatted pages, print and page exports, and SHOULD offer them in an editor beside the entry. They remain part of the source: editable exports, search within the editor and protection treat them like the rest of the entry, so a scratchpad is no more private than the entry it belongs to. An entry MAY have more than one; an editor that offers one scratchpad per entry uses the first and preserves the others. A scratchpad before the first dated heading belongs to no entry and is preserved. Choose a fence longer than any run of the same character inside the draft (3.2).

4.2.8 Margin notes on a passage

Marginalia (4.2) on its own line directly after a paragraph, list item or quote annotates that block. A renderer SHOULD place it beside the block, or after it on narrow pages, and it MUST NOT change the annotated text. Several notes MAY follow one block.

4.2.9 Where in a work, and the series it belongs to

A media reference (§4.2.3–4.2.5, or a named link to a catalog page) MAY be followed on the same line by up to three locators, each optionally after a comma or inside parentheses: pages (`p. 12`, `pp. 40–72`, `page 12`), chapters (`ch. 4`, `chapters 4–6`), an episode code (`S1E3`, `S1E3–E5`), tracks (`track 4`, `tracks 1–3`) or a time (`1:12:30`, `at 0:42:10`). A reader MAY show these exactly as written beside the work; anything else after the reference stays ordinary words. Locators say where the writer was, not how much is done: readers MUST NOT total them, turn them into progress or require them.

A reference MAY be followed by the series it belongs to, written as `(in [Series](link))`, for example `Watched [The Matrix Reloaded](imdb:tt0234215) (in [The Matrix series](https://www.wikidata.org/wiki/Q1210827))`. The series link belongs to that mention and is not a separate mention. A reader MAY group works by the series written this way, and an episode under the show its title begins with ("Severance S1E3: …"). What a reference names (a song or an album, an episode or a series, an edition or a work) is read from the reference itself (§4.2.5): readers MUST NOT infer it from a provider lookup or substitute one for the other. A tool that offers a series looks it up only when the writer asks, discloses the provider, and writes it only when the writer chooses it.

4.3 Generic structured record

```
:::record reflection #evening-20261025
method: gratitude-journaling@1
privacy: private
author: writer-a
gratitude: A friend made time for tea.
gratitude: The rain sounded gentle.
gratitude: I repaired something small.
body: |
  There was room for a difficult day and a good moment.
:::
```

A record is an ordered multimap of fields. Each field is `name: value` at column zero. Field names match the `name` grammar. Field keys are lowercase and case-sensitive. Everything after the first colon is a string with surrounding horizontal whitespace removed. Repeated fields represent repeated values in order. There are no implicit booleans or dates at this layer. Empty string, `0`, `false`, `unknown`, and `not-applicable` are different strings. Absence is different from all of them.

`name: |` introduces a multiline field: every subsequent line starts with two spaces, stripped exactly once; a blank content line is two spaces followed by a newline. It ends at the next non-indented line. Lines with `:::` can therefore be included as `  :::`. All markers inside a

multiline field are literal. Field values that need leading/trailing spaces or the literal value `|` use this multiline form. A malformed field line is retained in the record's `unparsed` content with a diagnostic. Empty records are Core-valid.

Reserved common fields are `method`, `privacy`, `author`, `created`, `modified`, `date`, `at`, `start`, `end`, `body`, `source`, `relation`, and `label`. A record MAY have several `method` fields. IDs are optional for ephemeral notes and required for durable linking, tasks under migration, attachments and cross-device reconciliation. Duplicate IDs within a document are errors; references to duplicates remain unresolved. Identical IDs in different documents are distinct.

5. Records, measurements and time

5.1 Common record types

The Core Reader recognizes a generic record regardless of its name. A Profile Processor interprets `task`, `event`, `reflection`, `prompt`, `response`, `measurement`, `scale`, `habit`, `observation`, `session`, `period`, `collection`, `quotation`, `attachment`, `layout`, `review`, `consent`, `medication`, `correction`, `relation`, `person`, `place`, `chart` (§5.2.1), `query` (§5.2.2), `sync` (§5.3.1), and `palette` and `moodboard` (§8.1.5). Applications MAY add namespaced types using an `x-organization-` prefix. A type name alone does not enforce a profile.

To make the profile contracts usable, every profile has an ID and integer version. A `method: id@1` field selects it. Every catalogue subsection lists its intended record type, required fields for a **complete** record, optional fields and validation notes. Core always accepts partial capture. Missing required profile fields produce a `profile-incomplete` warning, never rejection of the note. Unless explicitly repeatable, fields in a complete profile are singleton; duplicates warn and their last value supplies the effective value. `body`, `source`, `relation`, `label`, `author`, `privacy`, `created` and `modified` are available to every profile. Common record IDs are separate from profile fields.

5.1.1 People and places in reusable reference books

A `person` or `place` record is an ordinary generic record with a stable ID and a display `label`. The workspace's reference-book editor uses plain `.jot` files with conservative filenames (ASCII letters/digits, dots, underscores and dashes), and references records with ordinary relative links such as `[Robin](people.jot#person-robin)`. A record ID must stay stable when the label changes.

Recommended person fields: `label`, repeatable `alias`, `given-name`, `family-name`, `pronouns`, `organization`, repeatable `email`, `phone` and `website`, plus `address`, `place` and `notes`. Recommended place fields: `label`, repeatable `alias`, `kind`, `address-line1`, `address-line2`, `locality`, `region`, `postal-code`, `country`, `country-code`, `latitude`, `longitude`, `altitude-m`, `accuracy-m`, `timezone`, repeatable `phone` and `website`, and `directions`, `accessibility`, `notes`.

Coordinates are supplied WGS84 decimal degrees, latitude −90…90 and longitude −180…180, always paired. Elevation is optional metres (no vertical datum inferred); accuracy is a supplied nonnegative estimate in metres. Country codes are two letters; timezone names are IANA identifiers. A place timezone is descriptive only and MUST NOT implicitly change entry/document time. The editor checks these supplied fields, but Core does not infer coordinates, timezone, address deliverability or contact validity. Person/place numeric values MUST NOT become implicit measurements merely because they are numeric.

Loading a library is explicit. Reference selection must not copy contact details into journal text or exports. A reader may resolve a qualified link against opened files, but MUST NOT scan disks, fetch missing files or contact a map service automatically. The current workspace optionally remembers selected books in browser storage; portable journal packages do not implicitly include those books. Unknown fields and unrelated source must survive book editing. Plain reference books are not encrypted.

5.2 Measurements and scales

```
:::record scale #energy-scale
name: energy
kind: ordinal
minimum: 0
maximum: 5
anchor-low: No usable energy
anchor-high: Plenty for my usual day
:::

:::record measurement #energy-am
method: symptom-tracking@1
metric: energy
value: 2
scale: #energy-scale
at: 2026-10-25T08:30:00-07:00
status: observed
:::
```

A structured numeric value is an ASCII decimal without grouping, exponent or NaN: `-?[0-9]+(\.[0-9]+)?`. A measurement has `metric`, `value` and optionally `unit`, `scale`, `at`, `status`, `aggregation`, `precision` and `body`. `status` is `observed`, `estimated`, `unknown`, `skipped` or `not-applicable`. For the final three states, omit `value`. Profile contracts MUST allow this explicitly in measurement-based records. Zero is an observation, never a missing value. Repeated observations at different times remain separate records. A unit is an explicit string such as `h`, `min`, `mL`, `mg` or `oz`; no automatic conversion is permitted without a declared conversion and original-value retention. Identically named metrics with different scales MUST NOT be merged as comparable data.

A scale declares `name` and `kind` (`ordinal`, `numeric` or `categorical`). Numeric and ordinal scales declare `minimum` and `maximum`, with optional anchors. Categorical scales repeat `option` in display order. A reference to a missing scale is unresolved rather than defaulted. A missing measurement MUST remain visually distinct from the lowest value. Multiple mood observations on one day require a declared daily aggregation (`latest`, `mean`, `minimum`, `maximum` or `user-selected`) before a year-in-pixels view is generated; mean applies only to numeric scales. If no aggregation is chosen, display multiple markers or no single-day aggregate.

Inline state tags such as `[Energy: High]` are display annotations only. They MUST NOT become numeric measurements without an explicit import mapping. Legacy `Telemetry:` and `Theme-Check:` blocks require conversion into records. A free-form `telemetry` record MAY preserve source text while the author decides its mapping.

5.2.1 Charts

A `chart` record asks a reader to draw numbers the document already holds. It is a view, not data: it never carries values, and removing it loses nothing.

```
:::record chart #week-2026-10-26
metrics: sleep, steps
start: 2026-10-26
end: 2026-11-01
style: line
:::
```

`metrics` is a comma-separated list of metric keys; absent, every metric with a number in the range is charted. A metric's key is its measurement `metric`, or the name of any other record's field whose value is a structured numeric value (§5.2), lowercased with runs of other characters folded to `-` and a leading `x-` removed. Fields that describe a record rather than count something (the reserved common fields of §4.3, and `metric`, `value`, `unit`, `scale`, `status`, `name`, `kind`, `minimum`, `maximum`) are never metrics. `start` and `end` are inclusive dates. Absent, a chart inside a dated entry covers that entry's ISO week (Monday to Sunday); one outside any entry covers the dates that have numbers. A reversed range is read in date order. `style` is `line` (default) or `bars`.

A number's date is its record's own `date` (or the date part of `at`) when valid, else the dated entry containing it. Two numbers for one metric on one date are shown as their mean in a chart; this is a display choice and MUST NOT be written back as data. A day with no number MUST be drawn as a gap, never as zero (§5.2). Each metric is drawn against its own value axis: metrics with different units or scales MUST NOT share one axis. When a measurement names a scale, its `minimum` and `maximum` bound the axis, unless a value falls outside them. A

renderer SHOULD give each chart a text alternative (the metric, the range, how many days have numbers, the mean, the lowest and highest) and the numbers as a table. Readers that do not draw charts show the record as any other record.

5.2.2 Queries

A `query` record asks a reader to list or count what the same document already holds. Like a chart it is a view, not data: a reader computes it each time it is shown, never writes the result into the document, and removing it loses nothing. It is not a program: its fields are the fixed vocabulary below, and nothing in it is evaluated or fetched.

```
:::record query #open-health
label: Still to do for my health
show: tasks
status: open
tag: health
period: week
:::
```

Field	Values
`show` (required)	`tasks` (§4.1 rapid tasks), `events` (`o` events and `event` records), `entries` (dated entries, §2.3) or `numbers` (as §5.2.1 reads them, one per metric and date, two on one date as their mean)
`status`	tasks only: comma-separated `open`, `completed`, `migrated`, `scheduled`, `cancelled`; a task matching any of them
`priority`	tasks only: `high` or `normal`
`tag`, `context`	tasks, events and entries: comma-separated names, with or without `#` / `@`; an item carrying all of them (§4.1, compared case-insensitively). An entry carries the tags and contexts written anywhere in it outside records, comments and code; a task or event, those in its text including continuation lines (an `event` record, its `label`, `title` and `body`)
`text`	words the item contains, compared case-insensitively
`metric`	numbers only: comma-separated metric keys, folded as in §5.2.1
`period`	`day`, `week` (ISO, Monday to Sunday) or `month` containing the query's own `date` (or the date part of `at`) when valid, else the date of the dated entry it is in
`start`, `end`	inclusive dates; either alone leaves the other side open; a reversed range is read in date order
`stat`	`count` (default); for numbers also `sum`, `average`, `min`, `max`, computed for each metric separately
`limit`	the number of rows shown, 1 to 500 (default 50); counts and statistics still use every match

`label` titles the view; the other reserved common fields of §4.3 are allowed and do not filter. Without `period`, `start` or `end` every item counts, including undated ones; with any of them an item needs a date in the range. Items are listed in document order (numbers in date order).

A reader MUST NOT show a query it cannot fully read as an empty or unfiltered result. An unknown field, a repeated field, a filter field with an empty value, a value outside the vocabulary, a field that does not apply to the `show` (for example `status` with `numbers`, or `stat: sum` with `tasks`), `period` with `start` or `end`, `period` in an undated query outside any dated entry, an invalid date or a `metric` the document does not have, is a visible problem in place of the result and an `invalid-query` warning. A renderer SHOULD give each query a plain-text summary naming what was asked and how many matched (or each statistic and the number of days it covers), and the matches as a table. Readers that do not compute queries show the record as any other record.

5.3 Dates, times and duration

Dates use valid Gregorian `YYYY-MM-DD`. A complete instant uses an RFC 3339 timestamp with seconds and explicit offset or `Z`. Local timestamps lacking an offset MAY be stored as capture text; they remain unresolved unless an explicit date and validated timezone yield exactly one instant. A timezone field never resolves a repeated daylight-saving hour without an offset. An offset inconsistent with a declared IANA zone produces a warning; the written instant is preserved. A plain `HH:MM` is display-only unless a profile also supplies date and timezone and ambiguity is resolved explicitly.

Intervals use `start` and `end`; `end` is exclusive and MUST be later than start for a completed timed interval. Overnight intervals use full dates. Ongoing intervals omit `end` with `status: ongoing`. Durations use numeric `duration` and `duration-unit: min` or `s`; an inconsistent duration and interval is flagged, not silently corrected. Future dates do not imply reminders. Jot records schedules; actual notifications belong to an application feature with explicit user configuration.

`date` is the logical journal date; `at` is the observation instant; `created` is when it was entered. These MUST remain distinct. Backfilled material MAY use `recall: retrospective` and `confidence: uncertain` rather than pretend it was recorded contemporaneously. Prompt dates, planned dates, actual dates and import dates have separate fields.

5.3.1 Automation and synchronisation

Tools MAY read a document's tasks, events and measurements to export them (iCalendar, CSV), to import measurements, or to send them to a service the author configures. Such tools MUST write back only ordinary Jot: values in existing fields, new records carrying `source:` naming where they came from, task markers (for example `[>]` when a task is carried to another day). An event's clock time is taken only from an unambiguous expression (24-

hour `HH:MM`, or a 12-hour time with `am`/`pm`); otherwise the event is all-day. A tool that remembers what it has sent keeps that in a `sync` record (`:::record sync #sync-<service>`) with `service:`, an optional `last:` and one repeated `sent:` field per item; renderers SHOULD NOT display sync records. Nothing in a document triggers a network request by itself (§5.3).

An import from a health app's export (for example Apple Health's `export.xml`, or a daily CSV) MUST let the author choose the metrics and dates and show the records before writing them, and writes one `measurement` per metric per day with `source:` and an `aggregation` naming how the day's readings were combined: `sum` (a daily total, such as steps or hours asleep), `mean`, `minimum`, `maximum` or `latest`. Units are kept as the export states them; one metric exported in two units is written as two metrics, not converted (§5.2), except where the export's own unit definition requires it (HealthKit writes percentages as fractions: `0.97` with unit `%` is written `97`). Sleep is the time asleep per night, overlapping sources counted once, dated on the morning it ends. Importers MUST NOT read medications, laboratory or clinical records, reproductive or sexual health, or location by default, MUST NOT treat a vendor's calculated score as a measurement, and MUST NOT infer health states from the numbers (§9).

6. Tasks, collections and relationships

6.1 Task lifecycle

Rapid logs provide low-friction capture. Records provide durable identity and detailed behavior:

```
:::record task #repair-lamp
method: getting-things-done-paper@1
text: Replace the lamp cable
status: open
context: home
next-action: Photograph the connector label
project: #lamp-project
estimate: 15
estimate-unit: min
:::
```

Task status is `open`, `in-progress`, `waiting`, `scheduled`, `completed`, `cancelled` or `migrated`. `priority` is `low`, `normal` or `high`. `due` is a deadline; `scheduled-for` is a chosen date/time to work; they are not interchangeable. `completed-at` records completion. A scheduled task MUST have `scheduled-for` or remain profile-incomplete. A migrated task MUST have `migrated-to`, targeting the destination task or collection; unresolved targets are visible warnings. Marking a task migrated never marks it completed.

For an actual forward copy, create a new ID, retain `migrated-from` on the new task, and set `migrated-to` on the old. For a single task shown in a later daily view, retain the same ID and create a relation to that day instead of duplicating it. These are distinct migration modes. Automatic carry-forward MUST NOT silently choose one. Completed or cancelled tasks can be reopened only by an explicit edit or correction. A changed task status does not rewrite previous exports.

6.2 Collections and linked knowledge

A `collection` record declares `title` and `kind`, optionally repeating `member` links. `kind` is a user string such as `index`, `future-log`, `project`, `commonplace`, `inbox`, `waiting`, `someday`, `reference` or `insert`. Membership never moves or deletes the original. A generated index links headings and IDs; page numbers are output-specific locators and MUST NOT replace stable references. Cross-file references are relative to the current document, with a fragment naming a record ID. For a multi-document package, paths MUST stay within the package root.

An include is an image-style reference to another journal on a line of its own: `![label](other.jot#fragment)`. A reader MAY show the named part of that journal in place: the record with that ID, every dated entry with that date (`#2026-10-25`, up to the next dated heading), or, with no fragment, the whole body. Includes are view-only: the source keeps its one line, the included content is visibly framed and labelled with its source, and expanding includes into a copy (flattening) is a separate explicit operation that MUST NOT overwrite the original. Paths are relative and resolved against the including document; they MUST name a `.jot` file and stay within the root the reader declares (the workspace folder or the package root). Schemes, absolute paths, backslashes and `..` segments that climb above the root are `unsafe-path`, and a symbolic link MUST NOT lead outside the root. Includes are never fetched from a network. Readers MUST detect loops and SHOULD stop at a depth of 3; a missing file, an unknown fragment, an unsafe path, a loop or a depth limit shows a visible notice in place, never a silent blank. Content a reader cannot include (a `.jot` image within other words, for instance) is shown as a readable label.

Relations use `relation: kind target`, split on the first space. Standard kinds are `about`, `parent`, `continues`, `responds-to`, `derived-from`, `summarizes`, `revises`, `supports` and `contradicts`. Unknown kinds are preserved. Cyclic parent/migration chains warn; ordinary knowledge links MAY be cyclic. Unresolved references remain visible and MUST NOT prevent writing.

6.3 Reviews, summaries and recurring activity

A `review` record declares `period-start`, `period-end` (exclusive), `scope`, and repeated `source` references. Scopes include `day`, `week`, `month`, `quarter`, `season`, `year` and user labels. Highlights selected for a weekly review SHOULD link to their source entries. User conclusions and

generated summaries MUST be distinguishable via `origin: human`, `origin: imported` or `origin: generated`. Generated text MUST name its source selection and remain editable. Absence from a summary does not delete a source entry.

A `query` record (§5.2.2) is a view computed when shown, not a stored summary, so it needs no `origin`; text copied out of one into the document is generated text as above.

Core has no executable recurrence language. A profile MAY store `cadence: daily`, `weekly`, `monthly` or a human description, but MUST NOT infer missed tasks, fabricate observations, or schedule notifications. A habit observation is `done`, `partial`, `not-done`, `skipped`, `unknown` or `not-applicable`; only the first three describe participation. Missing days do not become `not-done`. Streak calculations, if offered, MUST state how partial/skipped days and denominator selection work and MUST be optional.

7. Method profiles and composition

Profiles are views and validation contracts over shared blocks. They MUST NOT require a new delimiter for each journaling method. Method identity is local to a record; a daily note can contain a sensory check, a work plan, a photo and free writing together. A file-level theme is a default label; it does not turn every paragraph into a theme score.

A profile definition consists of `id`, `version`, `label`, `record-type`, `required`, `optional`, `repeatable`, `constraints`, `suggested-view`, `source`, and `adaptation-notes`. Profiles in this document are version 1. A published revision that changes the meaning or required fields MUST increment the profile version. New optional fields MAY be added within a version if old readers preserve them. Custom methods SHOULD use `x-owner-name` IDs. Application templates are separate, versioned resources; a filled journal MUST retain the questions actually shown, not only a template ID that might later change.

Composition validates every selected profile independently. A combined record is complete only if every selected profile is complete and their field constraints are compatible. If profiles require incompatible record types or meanings, use separate linked records. For instance, put an emotion measurement beside a CBT reflection rather than overload `value` with both a rating and a narrative. An application MUST permit plain text capture even when a form is incomplete.

Prompt records declare `question`, optional `prompt-id`, `prompt-version`, `source`, `theme`, `offered-on`, and `content-note`. Response records use `prompt` as a link plus `answer` or `status: skipped`. A skipped response MUST NOT infer refusal, failure or a negative mood. Sensitive prompts and on-this-day resurfacing are opt-in presentation behavior. A prompt rotation MUST retain the presented wording and selection date, so later template changes do not rewrite history.

8. Presentation and optional page layouts

8.1 Content and layout separation

`@layout:`, `@spread:` and `@grid:` directives are presentation hints, scoped from their position to the next directive of the same name or EOF. `@hand:` and `@ink:` choose an entry's handwriting pen and ink (§8.3.1). Values are strings. They do not alter data, privacy or entry membership. Recognized values are `@layout: dot-grid-5mm`, `@layout: lined`, `@layout: blank`; `@spread: single-page` or `two-page`; and `@grid: normal`, `hidden` or `blackout`. Blackout is a dark visual background, never redaction. A renderer that cannot honor it MUST warn. Unknown values survive round trips.

More precise layouts use a `layout` record with `target`, `paper`, `orientation`, `margin-mm`, `grid-mm`, `gutter-mm`, `font-pt`, `palette` and repeated `panel` fields. Supported initial targets are `screen`, `print` and `eink`. Named and custom paper dimensions follow section 8.1.1. All lengths are positive decimal millimeters, except margins and gutter MAY be zero. Unsupported values produce a warning and an explicit fallback preview.

Visual journaling stores a text description, media link, optional crop and position, and physical-item locator. Layout is not required to express emotional meaning. Functional icons MUST have text labels or alternatives. Color MUST NOT be the only carrier of task state, mood, urgency or identity. Font, line spacing, grid pitch and density MUST be user-adjustable; a method does not mandate tiny writing.

8.1.1 Paper sizes and physical units

Paper size belongs to the rendering configuration. A `.jot` document containing a whole month MAY be rendered on A4, A5, Letter or a custom notebook size without changing its writing or dates. Physical pages MUST NOT determine entry boundaries. Choosing smaller paper reflows and paginates the selected writing; it MUST NOT automatically reduce handwriting space, grid spacing or font size to preserve the old page count.

The companion `paper-sizes.json` defines exact portrait dimensions in millimeters for ISO A0-A10 and ISO B0-B10, plus Letter, Legal, Tabloid, Half-Letter and Executive. ISO B and JIS B are different families; unqualified `B5` is not accepted as a canonical preset. Use `ISO-B5`, or define custom dimensions for another standard. Preset names are case-sensitive. `Ledger` is an accepted input alias for `Tabloid` with landscape orientation; writers emit the canonical size and explicit orientation. Unknown names remain preserved, but a renderer MUST request dimensions or present a clearly labeled fallback rather than silently substitute Letter.

Canonical preset	Portrait width x height (mm)	Typical use
A3	297 x 420	Large planning sheets

Canonical preset	Portrait width x height (mm)	Typical use
A4	210 x 297	Full-size journal pages
A5	148 x 210	Compact notebooks
A6	105 x 148	Small notebooks
A7	74 x 105	Small reference pages
ISO-B5	176 x 250	Intermediate notebook pages
ISO-B6	125 x 176	Compact notebook pages
Letter	215.9 x 279.4	US full-size pages
Legal	215.9 x 355.6	Longer US pages
Tabloid	279.4 x 431.8	Large US sheets
Half-Letter	139.7 x 215.9	US half-size pages
Executive	184.15 x 266.7	Intermediate US pages

For custom paper use `paper: custom` together with positive `width-mm` and `height-mm`. These dimensions describe the portrait base: width MUST be less than or equal to height. A landscape renderer swaps the two axes. A square is valid and unchanged by orientation. Named presets MUST NOT also declare width/height; conflicting dimensions produce a configuration error with no silent override. `orientation` is `portrait` or `landscape`, default portrait. A printer or renderer MAY support a subset of sizes, but MUST publish its limits and validate the selected size before export.

8.1.2 Simple authoring hints and optional detailed records

For common choices, readable hints are enough:

```
@paper: A5
@orientation: portrait
@margin: 12mm
@dot-spacing: 5mm
@dot-diameter: 0.5mm
@dot-color: #bbbbbb

### 2026-10-25

The writing stays the same when I choose another paper size.
```

`@paper` and `@orientation` use the preset and orientation vocabularies above. `@margin`, `@dot-spacing` and `@dot-diameter` accept a nonnegative decimal followed immediately by `mm` or `in`; dot spacing and diameter MUST be greater than zero. One inch is exactly 25.4 mm. `@dot-color` accepts a six-digit hexadecimal RGB color `#RRGGBB`. A renderer converts lengths to millimeters without discarding the original source. Dot diameter MUST be smaller than dot spacing. These directives apply from their position until another directive of the same name or EOF. A geometry change mid-page starts a new output page; it does not split a journal entry.

Detailed layouts use optional records. This custom-size example has the same 5 mm grid as the A5 example:

```
:::record layout #narrow-notebook
target: print
paper: custom
width-mm: 110
height-mm: 210
orientation: portrait
margin-mm: 10
grid-mm: 5
dot-diameter-mm: 0.5
dot-color: #bbbbbb
:::
```

An active print-layout record applies its explicit fields to subsequent writing until another print-layout record. For each property, renderer defaults have the lowest priority; author hints override defaults; fields in the active print-layout record override hints. An explicit export-dialog override has the highest priority and MUST be shown in the preview. Missing fields inherit lower-priority settings. No override rewrites source text without an explicit save operation. A newly selected named paper size clears inherited custom dimensions; a custom size requires both dimensions in the active configuration.

8.1.3 Margins, binding and grid anchoring

`margin-mm` sets all four page-edge insets. Optional `margin-top-mm`, `margin-right-mm`, `margin-bottom-mm` and `margin-left-mm` override individual edges. Insets are measured from the physical page, independent of a printer's imageable area. `binding` is `none`, `left`, `right` or `top`. `gutter-mm` adds space on that binding edge. `mirror-margins: true` swaps left/right margins and a left/right gutter on even numbered pages; odd pages are recto. The page size itself is unchanged. A two-page spread is a preview arrangement of two physical pages, not a doubled sheet size.

Usable width and height are the oriented sheet dimensions minus the applicable margins and gutter. Both MUST remain positive. The renderer MUST check that the chosen font, grid and selected content fit that region. A large margin on A7 can legitimately leave too little room; the renderer reports the problem rather than clipping the text or silently shrinking it.

`grid-area` is `writing-area` (default) or `full-page`. `grid-anchor` is `center` (default) or `top-left`, relative to the chosen area. Dot spacing measures center-to-center distance, independent of paper size. For `top-left`, the first dot center is one half-spacing from each area's starting edge; later centers advance by exactly one spacing. For `center`, place a centered lattice of complete dots with the same half-spacing minimum inset at the edges; any leftover space is divided equally. If the area is smaller than one spacing, it contains no complete grid cell and the renderer warns. Cropped edge dots MUST NOT be presented as full dots.

Dot diameter describes the full dot width, not the radius. A radial-gradient implementation therefore uses half that diameter for its radius. CSS pixel dimensions MUST NOT be used as the physical paper/grid contract: 20 CSS pixels is about 5.29 mm at the CSS reference conversion, not 5 mm. A 5 mm grid stays 5 mm on A4, A5, Letter and custom pages unless the user explicitly changes its pitch.

8.1.4 HTML, PDF and print validation

An HTML renderer SHOULD generate literal physical dimensions for the selected size, for example `@page { size: 148mm 210mm; margin: 0; }` plus a 148 x 210 mm sheet container. Margins then belong inside the sheet container; do not apply the same margins again through `@page`. The renderer may instead use nonzero page margins if its page-box model accounts for them exactly once. Browser viewport dimensions such as `100vw` and `100vh` are not paper sizes. Generate orientation-resolved literal values rather than depending on CSS custom-property substitution in `@page` across print engines.

PDF page boxes MUST match the requested sheet dimensions using 72 points per inch. Print at actual size/100% to preserve physical grid pitch; fit-to-page scaling changes it. Browser headers/footers, background printing and printer nonprintable edges remain separate settings. A renderer should offer a vector-dot PDF route when a browser cannot reliably print backgrounds. A full-page grid on a non-borderless printer may be cropped even when the source page is correct; show this limit in the preview when the printer capabilities are known.

Renderer acceptance cases: A4 portrait and landscape; A5 and A6; Letter and Half-Letter; ISO-B5 distinct from a custom 182 x 257 mm page; a 110 x 210 mm custom page; a square page; mirrored margins; a gutter; oversized margins; unsupported presets; a mid-document paper change; and invariant 5 mm dot pitch across all sizes. A physical calibration proof SHOULD include a labeled 100 mm line and a grid sample. Previewing successfully is not proof that a printer used the correct paper or scaling setting.

8.1.5 Palettes and moodboards

A `palette` record holds repeatable `color` fields, each `#RRGGBB` or the shorthand `#RGB` (case-insensitive; `#RGB` means `#RRGGBB` with each digit doubled), then optionally whitespace and a name: `color: #c0392b brick red`. `label` names the palette. Alpha (`#RRGGBBAA`) and color functions are not colors here. A field that does not match is preserved as written and produces an `invalid-color` warning on its line; a renderer shows a visible note in its place and never drops it silently. The same check applies to `color` fields in a `moodboard` record and in any record using the `moodboard-and-palette` profile; `color` fields elsewhere are ordinary strings.

A renderer draws each color as a swatch whose name and hex value are text on the swatch (color MUST NOT be the only carrier, §8.1), in whichever of black or white has the higher WCAG 2 contrast against it, which is always at least 4.5:1. Swatches print with their colors (print-color-adjust exact) and keep their text when backgrounds are not printed. Exports MAY write the colors as CSS custom properties on `:root` (names from the swatch name, letters and digits kept, otherwise `--color-N`, made unique with `-2`, `-3` …) or as JSON `{ name, colors: [{ name, hex, variable }] }`.

A `moodboard` record holds repeatable `image` (or `asset`) fields: a path to an attachment, then optionally a text description used as the picture's alternative text. `alt`, `caption` and `credit` fields after an `image` describe that picture. Other fields keep their order. A renderer draws only attached pictures (§9: no network request); a picture that is not attached shows its path and description. A paragraph containing only two or more Markdown images MAY be drawn as the same grid.

8.1.6 3D models

A `model` record shows an attached 3D model. It is data, never code: a reader draws it with its own viewer, and nothing in the journal runs.

```
:::record model #garden-bed
label: The garden bed in October
file: media/garden-bed.glb A clay model of the raised bed with three seedlings
poster: media/garden-bed.png
camera: 2, 1.5, 3
target: 0, 0.4, 0
lighting: soft
background: #f4efe4
animation: Grow
hotspot: 0.3, 0.9, 0.2 The first seedling
caption: Built from scraps on Sunday.
:::
```

`file` is the path of an attached glTF 2.0 model, then optionally whitespace and a description used as its text alternative: a binary `.glb` (`model/gltf-binary`), or a `.gltf` (`model/gltf+json`) whose buffers and images are embedded as `data:` URIs. A reader MUST NOT fetch any other URL a model names; it draws the model without that part and says so. Compressed models are ordinary glTF: `KHR_draco_mesh_compression` (Draco meshes), `EXT_meshopt_compression` with `KHR_mesh_quantization`, and `KHR_texture_basisu` (Basis Universal KTX2 textures), as well as `EXT_texture_webp` and `EXT_texture_avif` pictures. A reader decodes them with decoders it carries itself, never ones fetched for the occasion, and reads only the decoders a model's `extensionsUsed` names. A reader without a needed decoder shows the poster or description and says the model is compressed in a way it cannot read. `poster` is an attached still picture shown wherever the model is not drawn in 3D: in print, in a page export, in readers without 3D and before the reader asks for it. Without a poster, a reader shows the description. `camera` and `target` are three decimal numbers (`x, y, z`, comma or space separated, each within ±10,000) in the model's units; absent, the camera frames the whole model and looks at its middle. `lighting` is `soft` (default), `studio` or `dim`. `background` is `#RRGGBB` or `#RGB`; absent, the page shows through. `animation` names an animation in the model; a reader plays it, or the first animation when none is named, only when the reader asks. `hotspot` is repeatable, up to 20: three decimal numbers for a point on the model, then what is there. A renderer lists the hotspots as text wherever the model is shown, and while the model is drawn in 3D offers each as a control that brings the view to it. `label`, `caption` and `credit` are text. Other fields keep their order.

A field that does not fit is kept as written and produces an `invalid-model-field` warning on its line; a record without `file` produces `model-without-file`. A file that is not attached, or is not a glTF model, is reported in place of the model, never dropped silently (§8.3). Nothing moves until the reader asks: no automatic rotation, and drag, keys and buttons turn, zoom and reset the view. A renderer SHOULD keep the model's description as the accessible name of its drawing. Readers that do not draw models show the record as any other record.

A fixed-layout EPUB 3 export MAY draw models on its pages. Only the exporter's own viewer scripts are included, the pages that use them are declared `scripted`, the model's bytes, and the decoders a compressed model needs, are carried inside the book (never fetched), and each page keeps the poster or description for reading systems that do not run scripts (§9).

8.1.7 Flip cards

A `card` record is a question and its answer, for trying to recall before checking. It is plain text and needs no code to work.

```
:::record card #leaf
label: Biology, week 3
front: What does a leaf make from light, water and air?
back: |
  Sugar, and oxygen it lets go.
:::
```

`front` and `back` are text (the back may be multiline); `label` is optional. A renderer shows the front and keeps the back behind a control the reader uses to reveal it; nothing reveals it on its own, and nothing is scored, counted or scheduled. In print and on fixed-layout pages, where nothing can open, a card is a fold-over card: the front, then the back turned upside down below a fold line, still readable as text by assistive technology. A missing front or back produces `card-without-front` or `card-without-back` on the record. Other fields keep their order. Readers that do not draw cards show the record as any other record.

8.1.8 Sound and video recordings

An embedded attachment reference, `![Rain on the roof](media/rain.m4a)`, to an attached sound or video recording is drawn as a player for that file; the label is its accessible name. A renderer MUST NOT start playback, loop or play several recordings on its own: a recording plays only when the reader asks. It reads only the attached bytes (§9). In print and on fixed-layout pages laid out for print, a quiet box of the same size names the recording; a fixed-layout EPUB puts the player back in that box, carrying the file inside the book. A link (`[label](media/rain.m4a)`) stays a link to the attachment.

8.1.9 Places: a day drawn as a 3D scene

A dated entry MAY name a place: a small 3D scene of where the day happened, kept as an attachment. Places are **experimental**: the `jot-place` fields, limits and drawing may change before v1, and readers MUST keep ignoring what they do not draw.

```
### 2026-10-02 | Friday
@setting: media/7f3c2a…
Rain at the bus stop. The 42 was late again.
```

`@setting:` (a directive, §2.3) names an attached `jot-place` file (`application/vnd.jot.place+json`). It is presentation only: it never changes the writing, and readers that do not draw places ignore it. A place is data, never code: JSON with `format: "jot-place"`, `version: 1`, `made` (`model`, `at`), `title`, `sky` and `ground` colours, `weather` (`clear`, `rain`, `snow`, `fog`, `sun`), `time` (`morning`, `day`, `evening`, `night`), an optional `atmosphere` (`neutral`, `calm`, `cosy`, `bright`, `lively`, `tender`, `wistful`, `heavy`, `tense`; missing or unknown is `neutral`), a `camera` (`position`, `target`), and up to 120 `objects`, each a `name`, `position`, `rotation`, an optional `motion`

(`none`, `steam`, `flicker`, `sway`, `drift`, `turn`, `bob`; missing or unknown is `none`), the `moments` it holds and up to 12 `shapes` (720 in all): `box`, `cylinder`, `sphere`, `cone`, `torus`, `plane` or `capsule`, or a crafted part (`mug`, `teapot`, `plate`, `bottle`, `book`, `notebook`, `papers`, `laptop`, `phone`, `tv`, `lamp`, `candle`, `plant`, `tree`, `chair`, `table`, `bed`, `sofa`, `shelf`, `window`, `door`, `rug`, `bag`, `umbrella`, `bench`, `clock`, `frame`, `bus`, `streetlight`, `shelter`), with `position`, `rotation`, `size`, a `#rrggbb` colour, `roughness` and `glow`; a `plane` lies flat, sized by width and depth. A part is drawn by the reader from its own pieces, sized to its `size` as a bounding box centred on its `position`; its colour is its main colour. A reader that does not draw a part MAY draw its box instead. Coordinates are metres within ±50; sizes are 0.01–20. A reader clamps or drops what does not fit, as untrusted input.

`moments` are the entry's own sentences, word for word. A place maker that uses a model sends only the day's sentences, numbered, after the writer has seen exactly what goes and where; the model refers to sentences by number and never writes journal text, and the maker turns the numbers into the sentences before keeping the place. A reader shows a moment whose sentence is no longer in the entry as no longer there. Nothing is kept until the writer chooses to; keeping adds the attachment and one `@setting:` line under the entry's date heading. A place is drawn only with the reader's own code, and fetches nothing.

A reader MAY show the day's own media on a place's surfaces: on a `tv`, `laptop` or `phone` screen the title of something the entry says was watched, played or listened to; on a `book` cover the title of a book it is reading; in a `frame` a picture the entry itself attaches. Titles come from the entry's own references and pictures from its own attachments; nothing is fetched while reading. A kept place MAY carry up to 4 `stills`, each a `reference` (the media reference it shows, exactly as the entry writes it), a `path` (an attachment under `media/` holding the picture) and a `credit` (where the picture came from); a reader shows that picture on the place's screen (for a film or show) or on its book's cover (for a book) for that reference, and drops a still whose path is not a `media/` attachment. A maker adds stills only at the writer's choice; a model never supplies them.

`atmosphere` is how the place is drawn (its light, colour and pace), chosen by the maker from the feeling of the entry's words. It describes the place, not the writer: it is an interpretation shown as one, not a mood record, and readers MUST NOT collect, chart or summarize it across entries. `motion` names one of a fixed set of small movements the reader draws for a thing; there is no other animation data. A reader that animates a place MUST let the reader stop all movement and MUST keep it still when the reader prefers reduced motion. A place maker SHOULD depict only what the entry describes plus quiet setting details, and MUST NOT add people or events the entry does not contain.

8.2 Pagination and overflow

Renderers MUST preserve all selected writing. Overflow modes are `continue`, `additional-page` or `summary-with-source`. Clipping or silently shrinking text below the chosen minimum is forbidden. A renderer MUST show omitted content counts and continuation locators when a summary is explicitly chosen. Print output MUST declare paper size, scale, margins and page numbering. Privacy selection must remove excluded content from the exported document, not merely conceal it in a hidden layer.

8.3 Handwriting and scans

An attachment declares `path`, `media-type`, `alt`, optional `sha256`, `captured-at`, `page-label`, `region`, `physical-location` and `ocr-status`. `region` is a normalized `x,y,width,height` string with each decimal in [0,1] and the rectangle within the page. A transcription is a separate record linked with `derived-from`, with `origin: imported`, `review-status: unreviewed` or `verified`, and an optional confidence note. Handwriting recognition MUST NOT directly complete tasks, confirm doses or overwrite original values without review.

A `.jotz` is the portable bundle described in section 2.0. Handwriting images, editable strokes, scans, photos and other attachments live in UUID media members (or encrypted private members), resolved through the archive catalog from readable source links. Missing referenced assets must be reported; supplied but unused assets remain included until the user explicitly removes them. A plain `.jot` or a writing export is not a self-contained backup of a document with assets. Jot applications must save/restore the included assets along with writing and clearly distinguish an attachment download from downloading the whole document.

8.2.1 Formatted presentation

A renderer SHOULD present Jot's own constructs as a reader expects, without changing the source: a dated heading as its date in words (the text after `|` retained when it says more than the weekday), tasks with their lifecycle state (4.1) distinguishable without colour, event, question and idea markers as signs with text alternatives, tags and contexts marked, and marginalia beside or after the writing it annotates. Named presentation styles and exports (such as one standalone HTML file) are renderer features: they carry no data back into the document, MUST NOT fetch remote resources, and SHOULD embed what they show (charts as images with text alternatives, attachments, fonts).

8.3.1 Simulated handwriting

A renderer MAY present typed writing as handwriting: a handwriting font, alternate letter forms, small per-word rotation and offset, and ink colour. It is presentation only. It MUST NOT change the text, its order, selection, copying or accessible text, and SHOULD be

deterministic for the same text and seed so a reprint matches. Real handwriting (section 8.3) and simulated handwriting are never confused: simulation creates no attachment or stroke data.

Per-entry pen and ink

An entry can choose its own handwriting pen and ink without changing how earlier entries look:

```
### 2026-11-03 | Tuesday

@hand: script10
@ink: green

Today in the broad pen.
```

`@hand:` names a pen (`script05`, `script07`, `script10`, `pen`, `print` or `marker`; case, spaces, hyphens and underscores are ignored, so `Script-10` is `script10`). `@ink:` names an ink (`blue`, `black`, `green`, `purple`, `brown`) or a six-digit `#rrggbb` colour. `default` returns to the reader's own choice. Unlike the page directives of §8.1, which apply from their position, a `@hand:` or `@ink:` directive applies to the **whole entry that contains it**, its dated heading included, and to the entries after it, until another directive of the same name; directives in the writing before the first dated heading apply from the start. Entries before it are never restyled, so a later change of pen leaves the journal's past as it was drawn. An unrecognized value is kept as written and changes nothing. Pen and ink are presentation only: they apply when writing is presented as handwriting, never change the text, its order, selection, copying or accessible text, and carry no meaning a reader needs (§8.1: colour is never the only carrier).

A dot-grid presentation puts every line of writing on a row of dots: line height equals the dot pitch, block spacing and block heights are whole rows, larger text spans whole rows with its baseline on a row, and a printed page's writing area is a whole number of rows so pages stay aligned. The reference implementation snaps the pitch to the layout engine's unit (1/64 CSS px) so long documents do not drift.

8.4 E-ink and physical styles

E-ink views SHOULD minimize full-screen changes, use static navigation and outlines, and provide explicit refresh behavior. Exact display resolution, touch/stylus capability and contrast are device contracts rather than language rules. A monochrome preview is required before declaring a layout suitable for grayscale e-ink. Long notes need overflow navigation, not clipped boxes.

Dutch doors require an explicit persistent panel and per-leaf regions, plus collision and cut-guide checks. A digital pinned panel MAY approximate the interaction but MUST label itself a digital view. Traveler-style inserts map to collections and bundle members; they do not require leather, a fixed size or a brand. Sticker/deco and junk-journal assets preserve layering order and alternative descriptions. A dot grid is one selectable ruling, not a requirement of Jot.

9. Privacy, authorship and portability

Privacy is an intent label, not encryption or an access-control mechanism. A record's effective privacy is the more restrictive of the document and record (`private` > `shared` > `public`). A public record inside a private document therefore stays private for export. To publish an intentional selection, create a separate sanitized export with a declared target audience; never lower the source document's label silently. Local viewing by its owner is not publication.

A share/export command MUST let the user select content and include/exclude attachments, hidden comments, annotations, metadata and author identifiers. Default sharing excludes private content and hidden comments and strips precise locations. Excluded content MUST be removed from text, PDF layers, metadata, thumbnails, linked assets and manifests, not merely hidden with a black rectangle. The writing and assets inside a `.jot` document, including hidden comments, are accessible to anyone holding the unencrypted file. Encryption, password storage and secure deletion are application responsibilities, and applications MUST state their actual guarantees.

Collaborative writing records authors per block/record and uses separate `response` or `relation` records for replies. A person's words MUST NOT be silently reassigned to another author. Private sections need separate access-controlled storage; labels inside one shared document cannot conceal them. Conflicting edits MUST retain both versions, with a visible resolution step. Device modification times alone MUST NOT decide which journal text survives.

Export to Markdown, HTML, PDF, text, CSV or JSON MUST declare which constructs lose meaning. CSV exports require an explicit row model and selection, preserving units, scale IDs, missing statuses, source IDs and timestamps. HTML and PDF renderers MUST escape untrusted text and reject active content. Apps MUST NOT evaluate embedded code, expand shell expressions or execute template instructions found in a journal. An export that draws models (§8.1.6) MAY include the exporter's own viewer scripts; it MUST NOT include, generate or evaluate code taken from the journal. Search and AI features MUST respect the selected source scope; generated inference is never a measured observation.

10. Migration from Jot v1.0

The original PDF provided a syntax outline and an illustrative JSON object, not a full grammar or a published schema. Version 2 therefore has an explicit legacy import step, followed by a change report and preserved original.

Original outline	Expanded v1 draft
Initial unbounded key/value metadata	Wrap only the recognized opening metadata in `---` delimiters; retain uncertain lines as prose.
Fixed year/month/day heading levels	Retain levels; recognize dated entry headings at any level.
`[!]` urgent task	Preserve spelling; normalize to open plus high priority.
Bare `*` priority signifier	Convert only with explicit adjacency mapping; otherwise preserve text.
ALL-CAPS voices	Offer heuristic candidates; convert accepted ones to `::voice NAME`.
`LOG:`, `ENTRY:`, `INT.`, `EXT.` headings	Offer explicit scene conversion; no automatic Core recognition.
`.0830 - LOCATION`	Preserve as scene text; date/zone needed for a structured instant.
`Theme-Check:` and `Telemetry:`	Convert to records; retain original strings and scale descriptions.
`[Energy: High]`	Preserve display tag; numeric mapping is opt-in.
`[[ ... ]]`	Keep annotation meaning; do not reinterpret as wiki links.
Inline `/* ... */`	Convert to standalone hidden-comment block or literal text with report.
`===`, emphasis, layout directives	Preserve with the v1 recognition rules; report changed edge cases.
Example remote `$schema` URL	Replace with bundled schema reference; do not assert a hosted registry exists.

A migration MUST NOT invent dates, task targets, health units, timestamps, author identities or ratings. An original-outline scheduled/migrated marker without a target remains incomplete. Reversible import includes original file hash, selected source dialect, importer version and a list of transformations. No source file is deleted after conversion.

11. AST and validation contract

11.1 Lossless and semantic layers

The normative interchange is a source-preserving JSON envelope. The companion `jot-ast.schema.json` validates that envelope and structural node shapes. It does not validate clinical instruments, dates, cross-references or every profile. Those are semantic validation rules in this document. A structural schema success MUST NOT be reported as full journal conformance.

The root has `format: jot`, `version: 1.0`, original `source`, ordered `metadata`, ordered flat `nodes`, and ordered `diagnostics`. Each node has `type`, byte `start`, byte `end` and `raw`. Record nodes also have `recordType`, optional `id`, ordered `fields`, and `unparsed`. Field objects have `name` and decoded string `value`. Heading nodes have `level` and `text`; paragraph and other text blocks have `text`. A derived entry index MAY reference node ranges, but MUST NOT duplicate or replace source ownership. A link target node MAY carry `anchor` (its record ID or entry date, §4.2.2), and a derived `links` list MAY give each internal link's `href`, `target`, optional `file` and `field`, byte `start`/`end`, the index of the `node` holding it and, when it resolves, the index `to` of its target. Semantic caches and views are rebuildable.

Every node's `raw` MUST equal the UTF-8 slice `source[start:end]`. Nodes are contiguous, non-overlapping and cover the document body after the metadata region, including blank-line nodes. Metadata includes its own source range in `metadataRange` when present; without metadata, coverage starts at byte zero. The AST retains every byte, including line endings, unknown blocks and malformed content. Normalized semantic edits may change bytes; untouched nodes MUST remain identical in lossless mode. Canonical formatting is a separate explicit operation.

Diagnostics have `severity` (`info`, `warning`, `error`), stable `code`, `message`, `start` and `end`. Minimum codes: `unsupported-version`, `unclosed-fence`, `unclosed-record`, `unclosed-comment`, `invalid-field`, `invalid-date`, `duplicate-id`, `duplicate-singleton`, `unresolved-reference`, `ambiguous-reference`, `profile-incomplete`, `profile-conflict`, `invalid-measurement`, `ambiguous-local-time`, `unsafe-path`, `missing-asset`, `layout-overflow` and `invalid-color` (§8.1.5).

11.2 Semantic validation checklist

Validate Gregorian dates, timestamp offsets, range order, profile ID/version availability, profile required fields, numeric/scale bounds, categorical membership, repeatability, task lifecycle requirements, IDs and links. Missing profile definitions are warnings and opaque preservation, not syntax errors. Reject unsafe bundle paths before reading assets. Unknown fields remain valid unless they violate a declared profile constraint. Validators SHOULD provide actionable messages that point to the original line and preserve unsaved text.

Resource limits MUST be published. A Core Reader SHOULD support at least 1 MiB of text, 64 KiB lines and 10,000 records; smaller devices MAY stream or page larger documents. Limits MUST cause a visible partial/read-only state rather than truncation or successful-save claims. There are no remote includes or automatic expansion features: local includes (6.2) are explicit, view-only and bounded, and flattening them into a copy happens only when someone asks. Parsing SHOULD run in linear time in source length using bounded delimiter scanning.

12. Conformance fixtures and acceptance criteria

The following cases are normative input/expected-behavior pairs. They form a starter conformance suite, not evidence of an implemented parser. The companion `conformance.json` makes them machine-readable.

Case	Expected behavior
A single prose sentence, no metadata/date	Valid undated paragraph.
`[!] Phone the repair shop`	Open task, high priority.
`[x]` without following space/text	Ordinary prose.
`I AM TIRED` between blank lines	Prose, no voice inference.
`[[Remember the spare key]]`	Annotation, no link.
`[earlier](#day-am)`	Internal link, unresolved if ID absent.
`[then](#2026-10-25)` beneath that dated heading	Link to the entry's first heading with that date.
`[there](other.jot#a)`	Link to another document; kept, not opened or flagged.
`[ ] literal` inside a code fence	Literal, no task.
Escaped `\[ ] literal`	Prose showing `[ ] literal`.
Two headings with the same date	Two ordered entries.
`### 2026-02-30`	Heading retained; invalid-date diagnostic.
Measurement value `0`, minimum `0`	Observed zero, not missing.
Measurement status unknown, absent value	Unknown, not observed zero.
Closed record with unknown type/field	Valid opaque preservation.
Missing closing record delimiter	Raw retained through EOF; error.
Duplicate record IDs	Error; references unresolved.

Case	Expected behavior
Migration marker with no target	Capture valid; lifecycle incomplete warning.
Local time in a repeated DST hour, no offset	Unresolved instant; no guessed offset.
Attachment `../private.txt` in a bundle	Unsafe path rejected.
Export excluding private record	No private bytes in output or packaged assets.
Source with CRLF and trailing spaces	Lossless round trip is byte-identical.

Before claiming an implementation complete: exercise those cases plus empty files, BOMs, emoji byte offsets, multiline empty values, malformed fields, unmatched inline delimiters, duplicate metadata, nested opener recovery, overnight sessions, leap days, missing assets, out-of-scale ratings, incompatible profiles, interrupted saves and conflicting edits. A renderer additionally needs grayscale, large type, narrow screen, Letter/A4 overflow and export-redaction checks.

13. Method catalogue and worked examples

The following catalogue maps all repository method/style guides to Jot profiles and adds complementary practices. All sample field names are part of the proposed profile contracts. Fields listed as required describe a complete profile record; partial notes remain valid Core. Each example is one record that can be pasted into a `.jot` document. `source` links in the final register locate the repository guide; they are not a claim that Jot exactly implements a branded form.

13.1 ADHD journaling and planning

Profile: `adhd-journaling@1` | Record type: `reflection`

Externalize the next action, time estimate and what actually happened. Allow a tiny restart after any gap, without requiring backlog cleanup.

Writing invitation: What is enough for today? What is the smallest visible first step?

Complete structured record: `next-step`. Optional fields: `estimate`, `actual`, `done`, `distraction`, `restart`, `support`.

A done list and an interruption parking lot are optional. No automatic failure score, compulsory streak, or assumption that an unused planner means low motivation.

```
:::record reflection #example-adhd-journaling
method: adhd-journaling@1
next-step: Put the notebook beside the kettle
estimate: 2 min
done: Opened the notebook
:::
```

13.2 Alastair lists and rolling columns

Profile: `alastair-method@1` | Record type: `task`

Keep one task list and associate items with month, day, context or project-status columns. The columns are a view over items, so an item need not be rewritten for every column.

Writing invitation: Which column helps me find this again: month, day, context, or status?

Complete structured record: `text`, `status`, `column`. Optional fields: `column-kind`, `scheduled-for`, `migrated-to`.

column is repeatable for contexts/days. A project-status view uses a single current status. A month column is not an exact appointment date; retain the exact date separately when known.

```
:::record task #example-alastair-method
method: alastair-method@1
text: Book a repair appointment
status: open
column: November
column-kind: month
:::
```

13.3 Art, junk and sketchbook journals

Profile: `art-and-junk-journaling@1` | Record type: `reflection`

Record a day through drawings, collage, found paper and a few words. The journal can describe a physical page while keeping scans and transcriptions as optional companions.

Writing invitation: What image, scrap, texture or mark belongs to this day?

Complete structured record: `body`. Optional fields: `asset`, `physical-location`, `material`, `layer`, `alt`.

asset, material and layer are repeatable. Physical objects need not be digitized. Preserve layer order and text alternatives; do not claim that a text record reproduces the texture of an original page.

```
:::record reflection #example-art-and-junk-journaling
method: art-and-junk-journaling@1
body: A bus ticket beside a sketch of wet boots.
physical-location: Blue notebook, page 18
:::
```

13.4 Sensory and autistic experience logs

Profile: `autistic-sensory-journaling@1` | Record type: `observation`

Notice sensory context, capacity, communication preferences and supports. Entries can be short words or icons with text labels, without turning regulation into compliance.

Writing invitation: What was the environment like? What helped? What do I need next?

Complete structured record: `context`, `support`. Optional fields: `sensory-load`, `communication`, `masking`, `recovery`, `body`.

support is repeatable. Ratings need an explicit personal scale. A log describes experience; it must not infer a diagnosis, demand reduced stimming or automatically recommend increasing sensory tolerance.

```
:::record observation #example-autistic-sensory-journaling
method: autistic-sensory-journaling@1
context: Busy grocery aisle
support: Headphones and a quieter checkout
communication: Typing preferred
:::
```

13.5 Bullet Journal and rapid logging

Profile: `bullet-journal@1` | Record type: `collection`

Combine tasks, events, notes, a future log and topic collections. Reviews deliberately decide what to carry forward, schedule or release.

Writing invitation: What happened, what needs attention, and what is still worth carrying?

Complete structured record: `title`, `kind`. Optional fields: `member`, `period-start`, `period-end`.

Use ordinary rapid-log lines for everyday writing. member is repeatable when building an index. Migration follows section 6; an unfinished item is not automatically copied or counted as a failure.

```
:::record collection #example-bullet-journal
method: bullet-journal@1
title: October daily log
kind: daily-log
:::
```

13.6 CBT-style thought records

Profile: `cbt-thought-record@1` | Record type: `reflection`

Separate a situation, an automatic thought, feelings, observations and an alternative interpretation. Preserve the original wording instead of silently reframing it.

Writing invitation: What happened? What did I think? What evidence can I actually observe?

Complete structured record: `situation`, `thought`, `emotion`, `evidence-for`, `evidence-against`, `alternative`. Optional fields: `before`, `after`, `scale`, `action`, `outcome`.

evidence-for and evidence-against are repeatable. Before/after ratings require the same declared scale. This is a recording adaptation, not a validated replacement for a particular clinical worksheet; software must not decide that a reported harm is a distorted thought.

```
:::record reflection #example-cbt-thought-record
method: cbt-thought-record@1
situation: A message had no reply this afternoon.
thought: Perhaps I annoyed them.
emotion: Worry
evidence-for: I have not received a reply.
evidence-against: They said this week was busy.
alternative: I do not know yet; there are several explanations.
:::
```

13.7 Commonplace books and linked ideas

Profile: `commonplace-book@1` | Record type: `quotation`

Collect passages, observations and ideas under subjects, with clear attribution and commentary. Subject organization can coexist with chronological diary writing in the same file.

Writing invitation: What is worth keeping, where did it come from, and what does it mean to me?

Complete structured record: `text`, `attribution`. Optional fields: `source`, `locator`, `topic`, `commentary`, `relation`.

topic and source are repeatable. Quotations and personal commentary remain distinct. Links can connect ideas without claiming that a commonplace collection is identical to a full Zettelkasten system.

```
:::record quotation #example-commonplace-book
method: commonplace-book@1
text: Start with the thing you can inspect.
attribution: My own workshop note
topic: Making
commentary: A useful reminder before debugging.
:::
```

13.8 DBT-style diary cards

Profile: `dbt-diary-card@1` | Record type: `observation`

Keep brief, dated records of emotions, target behaviors and skill practice. A weekly card is a view assembled from observations, not a reason to flatten all daily data into one score.

Writing invitation: What did I notice, and what skill or support did I try?

Complete structured record: `date`, `target`, `status`. Optional fields: `value`, `scale`, `skill`, `effect`, `body`.

Use locally configured fields and scales from the actual care plan. No automatic clinical risk classification or unsolicited disclosure. Skill-use and urge ratings must not be conflated. This is an optional representation, not certification of a clinical diary card.

```
:::record observation #example-dbt-diary-card
method: dbt-diary-card@1
date: 2026-10-25
target: Skills practice
status: observed
skill: Grounding with a familiar object
body: It helped me return to the conversation.
:::
```

13.9 Daily notes and periodic reviews

Profile: `digital-daily-notes@1` | Record type: `reflection`

Use date headings, links and optional prompts as navigation. A daily note is a section of writing; a month file and separate day files are equally valid storage choices.

Writing invitation: What should future me be able to find from today?

Complete structured record: `date`, `body`. Optional fields: `template-id`, `template-version`, `relation`.

Do not require a file per date or infer dates from filenames. Automatic templates must leave the actual questions in the writing. Periodic summaries link to sources and preserve unsummarized material.

```
:::record reflection #example-digital-daily-notes
method: digital-daily-notes@1
date: 2026-10-25
body: I learned why the connector was loose.
:::
```

13.10 Dream journals

Profile: `dream-journal@1` | Record type: `reflection`

Capture remembered scenes soon after waking, allowing fragments and uncertainty. Optional interpretation belongs in a separate field from the remembered report.

Writing invitation: What fragments, feelings or images do I remember?

Complete structured record: `report`. Optional fields: `woke-at`, `recall`, `emotion`, `dreamsign`, `interpretation`.

emotion and dreamsign are repeatable. A blank report can be a partial record, or status unknown in a generic observation. Interpretation is the writer's reflection, never a factual inference about other people or a diagnosis.

```
:::record reflection #example-dream-journal
method: dream-journal@1
report: A train station with plants growing through the roof.
recall: Fragments
emotion: Curious
:::
```

13.11 Dutch door pages and persistent panels

Profile: `dutch-door-method@1` | Record type: `layout`

Keep a weekly or reference panel visible while daily leaves change underneath. This is a physical/digital layout style that can present any journaling method.

Writing invitation: What information should remain in view while I turn the page?

Complete structured record: `target`, `persistent-panel`, `leaf-region`. Optional fields: `paper`, `cut-guide`, `member`.

The human-readable region descriptions in this profile are design intent, not cut-ready coordinates. A production renderer must resolve geometry, check safe regions and provide a proof before calling the result printable.

```
:::record layout #example-dutch-door-method
method: dutch-door-method@1
target: print
persistent-panel: Weekly appointments at top
leaf-region: Daily notes below
:::
```

13.12 E-ink journal layouts

Profile: `e-ink-templates@1` | Record type: `layout`

Present writing in calm pages with readable contrast, explicit navigation and room for handwriting. Match the actual device rather than assume every e-ink screen has the same input.

Writing invitation: What should be visible at a glance, and where can longer writing continue?

Complete structured record: `target`, `palette`. Optional fields: `width-px`, `height-px`, `input`, `overflow`, `refresh`.

Pixel dimensions are positive integers. Navigation remains available without animation or color. Template rendering is separate from the .jot source, and handwritten originals stay attached when text is recognized.

```
:::record layout #example-e-ink-templates
method: e-ink-templates@1
target: eink
palette: monochrome
overflow: continue
input: stylus
:::
```

13.13 Urgency and importance sorting

Profile: `eisenhower-matrix@1` | Record type: `task`

Sort tasks on two independent dimensions and choose an action deliberately. A deadline and a personal priority are different properties.

Writing invitation: Is it time-sensitive? Does it matter to my values or commitments?

Complete structured record: `text`, `status`, `urgent`, `important`. Optional fields: `due`, `decision`, `delegate`.

urgent and important are explicit true/false strings; absence means unclassified. Both true suggests do; important only suggests schedule; urgent only suggests delegate/batch; neither suggests release/reconsider. Suggestions never delete tasks.

```
:::record task #example-eisenhower-matrix
method: eisenhower-matrix@1
text: Prepare questions for the appointment
status: open
urgent: false
important: true
decision: schedule
:::
```

13.14 Emergent daily planning

Profile: `emergent-task-planner@1` | Record type: `reflection`

Choose a few important tasks, estimate time and record what emerges. Let interruptions and changing capacity remain part of the day's account.

Writing invitation: What are my few important outcomes, and what actually happened around them?

Complete structured record: `focus`. Optional fields: `estimate`, `actual`, `interruption`, `available-time`.

focus and interruption are repeatable. Time bubbles are a display choice; they must label their duration. A custom layout is an adaptation, not a reproduction of the official planner artwork.

```
:::record reflection #example-emergent-task-planner
method: emergent-task-planner@1
focus: Inspect the power cable
estimate: 15 min
actual: 25 min
interruption: Needed a different screwdriver
:::
```

13.15 Expressive writing sessions

Profile: `expressive-writing@1` | Record type: `session`

Provide private, uninterrupted prose with an optional time boundary and a gentle closing note. The writing can remain entirely unstructured.

Writing invitation: What do I want to put into words, and what would help me close this session?

Complete structured record: `body`. Optional fields: `start`, `end`, `duration`, `duration-unit`, `closing`.

A saved session does not require disclosure, analysis, a minimum word count or an escalating prompt. Duration settings are user choices. Claims about a particular therapeutic protocol require its own evidence and qualified implementation.

```
:::record session #example-expressive-writing
method: expressive-writing@1
body: I want to describe what the change has felt like.
duration: 10
duration-unit: min
closing: Tea and a quieter room
:::
```

13.16 Short morning and evening reflection

Profile: `five-minute-journal@1` | Record type: `reflection`

Use a small morning intention and an evening look-back. Generic, original prompts can support a brief routine without copying a commercial page.

Writing invitation: One good thing, one intention, and one thing I noticed by evening.

Complete structured record: `phase`, `body`. Optional fields: `gratitude`, `intention`, `affirmation`, `highlight`, `adjustment`.

phase is morning or evening; missing either is not a failure. gratitude and highlight are repeatable. No forced positivity, quota or requirement to reframe a difficult day as a good one.

```
:::record reflection #example-five-minute-journal
method: five-minute-journal@1
phase: morning
body: I want a slower start.
gratitude: A warm room
intention: Leave time to eat
:::
```

13.17 Multi-year and line-a-day diaries

Profile: `five-year-diary@1` | Record type: `reflection`

Write a small entry on a real date, then compare the same month/day across years. The multi-year page is a view over ordinary dated writing.

Writing invitation: What small detail from today might I be glad to rediscover?

Complete structured record: `date`, `body`. Optional fields: `prompt`, `memory-visibility`.

Resurfacing is opt-in and excludes private material from shared views. Actual dates are retained; February 29 stays February 29. A short display limit must not truncate the stored writing.

```
:::record reflection #example-five-year-diary
method: five-year-diary@1
date: 2026-10-25
body: Rain on the window while I fixed the lamp.
:::
```

13.18 Goals, weekly outcomes and daily focus

Profile: `full-focus-planner@1` | Record type: `reflection`

Connect a longer-term goal to a weekly outcome and a small daily selection. Add startup/shutdown rituals only when they help.

Writing invitation: Which daily action supports this week's outcome?

Complete structured record: `horizon`, `outcome`. Optional fields: `goal`, `next-action`, `ritual`, `review`.

horizon is day, week, quarter or year. outcome and ritual are repeatable. A top-three view is a configurable preference, not a Core limit. This profile borrows a planning pattern and does not reproduce a branded planner.

```
:::record reflection #example-full-focus-planner
method: full-focus-planner@1
horizon: day
outcome: Understand the cable fault
next-action: Check the printed connector label
:::
```

13.19 Gender euphoria and HRT records

Profile: `gender-euphoria-and-hrt-log@1` | Record type: `observation`

Give moments of rightness, identity and joy their own space. Medication and laboratory records can sit nearby without reducing the journal to a treatment timeline.

Writing invitation: When did I feel like myself? What would I like to remember?

Complete structured record: `category`, `body`. Optional fields: `at`, `medication`, `lab`, `source`.

Keep joy observations separate from medication dose events and lab measurements. No automatic dose suggestions, assumptions about anatomy, or requirement to record dysphoria. Names and identity metadata require careful export selection.

```
:::record observation #example-gender-euphoria-and-hrt-log
method: gender-euphoria-and-hrt-log@1
category: euphoria
body: I liked hearing my name in a familiar voice.
:::
```

13.20 Capture, next actions and weekly review

Profile: `getting-things-done-paper@1` | Record type: `task`

Capture an open loop, clarify its next visible action and organize it by context, project or waiting state. Reviews reconnect lists to current commitments.

Writing invitation: What is this, and what is the next physical action?

Complete structured record: `text`, `status`, `next-action`. Optional fields: `project`, `context`, `waiting-for`, `review-on`, `scheduled-for`.

A project may contain several linked tasks; waiting-for is distinct from delegated ownership. The calendar holds explicit commitments rather than guessed deadlines. Reference material remains separate from actions.

```
:::record task #example-getting-things-done-paper
method: getting-things-done-paper@1
text: Repair the lamp
status: open
next-action: Photograph the connector
context: home
:::
```

13.21 Gratitude and small good things

Profile: `gratitude-journaling@1` | Record type: `reflection`

Name a specific thing appreciated, optionally including why it mattered. An entry can hold gratitude and discomfort together.

Writing invitation: What was a small good thing, without needing the whole day to be good?

Complete structured record: `gratitude`. Optional fields: `why`, `person`, `body`.

gratitude is repeatable. No minimum count or mandatory daily cadence. Avoid making a gratitude entry a condition for saving other writing, and let the writer skip the prompt.

```
:::record reflection #example-gratitude-journaling
method: gratitude-journaling@1
gratitude: Someone saved me a seat.
why: I felt welcome.
:::
```

13.22 Habits and partial participation

Profile: `habit-tracking@1` | Record type: `habit`

Record participation in a behavior with room for partial, skipped and unknown states. A chosen cue can be recorded without claiming that tracking proves a habit is automatic.

Writing invitation: What did I try, and what made it easier to begin?

Complete structured record: `name`, `date`, `status`. Optional fields: `cue`, `amount`, `unit`, `body`.

status uses the habit vocabulary in section 6. Amount is optional numeric data with an explicit unit. Do not treat partial participation, illness, missing data and a deliberate rest day as the same state.

```
:::record habit #example-habit-tracking
method: habit-tracking@1
name: Read a little
date: 2026-10-25
status: partial
body: One page before bed
:::
```

13.23 Daily life-book pages

Profile: `hobonichi-techo@1` | Record type: `reflection`

Mix a dated day page, small timeline, free writing, sketches and ephemera. The style gives the day room without imposing a single method.

Writing invitation: What belongs on this day's page: words, a sketch, a receipt, or nothing yet?

Complete structured record: `date`, `body`. Optional fields: `asset`, `layout`, `timeline`.

A brand-inspired daily view must not imply an exact replica of a commercial page. Grid pitch, paper size and decorative elements are user choices; overflow preserves long entries.

```
:::record reflection #example-hobonichi-techo
method: hobonichi-techo@1
date: 2026-10-25
body: The new tea tasted like oranges.
:::
```

13.24 Interstitial and transition notes

Profile: `interstitial-journaling@1` | Record type: `reflection`

Write a few lines when leaving one activity and entering the next. Preserve where to resume as well as what happened.

Writing invitation: Where did I stop? How am I arriving? What is the first next action?

Complete structured record: `stopped`, `next-step`. Optional fields: `at`, `feeling`, `resume`.

A scene heading can supply a human time cue; an exact instant needs at with offset. A transition note does not claim that the previous task is completed.

```
:::record reflection #example-interstitial-journaling
method: interstitial-journaling@1
stopped: Measured the connector spacing
next-step: Compare the photo with the parts list
resume: Notebook page 18
:::
```

13.25 Short ordered next-day lists

Profile: `ivy-lee-method@1` | Record type: `collection`

Choose a small ranked list for the next work period and revisit it deliberately. Carry-forward keeps task identity and distinguishes unfinished work from newly selected work.

Writing invitation: What few things matter most tomorrow, and in what order?

Complete structured record: `title`, `kind`. Optional fields: `for-date`, `member`, `limit`.

member is repeatable and order is meaningful. The traditional six-item limit can be a view setting; a smaller capacity-based list is an explicitly labeled adaptation. Reordering does not change deadlines.

```
:::record collection #example-ivy-lee-method
method: ivy-lee-method@1
title: Tomorrow
kind: ranked-actions
for-date: 2026-10-26
limit: 6
:::
```

13.26 Lifelog plus lasting reference

Profile: `jibun-techo@1` | Record type: `collection`

Separate day/week records from lasting life reference and loose ideas. A 24-hour view can show both planned and actual activity.

Writing invitation: Is this for the current diary, lasting reference, or an idea I want to keep?

Complete structured record: `title`, `kind`. Optional fields: `member`, `horizon`, `layout`.

A collection need not be a separate file. Distinguish plan and actual intervals; keep overnight activity on actual dates. Do not copy brand artwork or imply exact page compatibility.

```
:::record collection #example-jibun-techo
method: jibun-techo@1
title: Things I want to learn
kind: life-reference
horizon: ongoing
:::
```

13.27 Mood observations and year views

Profile: `mood-tracking-year-in-pixels@1` | Record type: `measurement`

Keep a mood label or rating with context, then generate a yearly view only from an explicit daily selection or aggregation rule.

Writing invitation: Which word or rating describes this moment, on my own defined scale?

Complete structured record: `metric`, `status`. Optional fields: `value`, `scale`, `at`, `aggregation`, `body`.

A categorical scale may be inline human documentation or a linked scale record; automated comparison requires a linked scale. value is required when observed or estimated. A year view must label missing days and retain multiple daily observations.

```
:::record measurement #example-mood-tracking-year-in-pixels
method: mood-tracking-year-in-pixels@1
metric: mood
status: observed
value: calm
body: A word label, not a diagnostic score.
:::
```

13.28 Morning Pages and freewriting

Profile: `morning-pages@1` | Record type: `session`

Write continuously in ordinary paragraphs, without judging style or solving everything. Longhand and a three-page practice can be tracked as a personal ritual without becoming a file constraint.

Writing invitation: Start with whatever is present, including not knowing what to write.

Complete structured record: `body`. Optional fields: `date`, `medium`, `page-count`, `closing`.

A typed or shorter session is a labeled adaptation if the writer wants to distinguish it from the original practice. No automatic sentiment scoring, summary or resurfacing of private freewriting.

```
:::record session #example-morning-pages
method: morning-pages@1
body: I woke up with three unrelated ideas and no need to sort them yet.
medium: handwriting
:::
```

13.29 Paper, ruling and page geometry

Profile: `paper-and-grids@1` | Record type: `layout`

Choose blank, lined, square, dot or cue-column ruling to fit the writer and output. Layout records preserve choices without altering the words.

Writing invitation: What spacing lets me write comfortably?

Complete structured record: `target`, `ruling`. Optional fields: `grid-mm`, `line-mm`, `paper`, `margin-mm`, `font-pt`, plus the paper, orientation, dot, binding and per-edge margin settings in section 8.1.

ruling is blank, lined, square, dot or cornell. Positive lengths and usable-area checks are required before rendering. Printer performance and paper behavior need an actual proof; the format cannot guarantee physical ink behavior.

```
:::record layout #example-paper-and-grids
method: paper-and-grids@1
target: print
ruling: dot
grid-mm: 5
paper: Letter
margin-mm: 12
:::
```

13.30 Roadmaps and values-led planning

Profile: `passion-planner@1` | Record type: `reflection`

Connect a future possibility to a concrete next step, with space for personal and work focus and a monthly look-back.

Writing invitation: What would I like more of, and which step would bring it closer?

Complete structured record: `horizon`, `aspiration`, `next-step`. Optional fields: `domain`, `milestone`, `due`, `not-to-do`, `review`.

milestone and not-to-do are repeatable. Aspirations are not obligations, and a calendar date is not inferred from a horizon phrase. This is a general roadmap adaptation, not a reproduced commercial worksheet.

```
:::record reflection #example-passion-planner
method: passion-planner@1
horizon: three months
aspiration: Make more small creative things
next-step: Reserve one quiet evening
:::
```

13.31 Functional stickers and decorative journaling

Profile: `planner-sticker-and-deco-culture@1` | Record type: `attachment`

Let decoration carry memory or aid navigation, while functional stickers can label a section or event. Treat artwork as optional rather than a prerequisite for using the journal.

Writing invitation: Is this decoration, a memory, or a symbol that carries information?

Complete structured record: `path`, `media-type`, `alt`, `role`. Optional fields: `relation`, `layer`, `license`, `creator`.

role is decorative or functional. Functional assets need a text equivalent; decorative assets may have a brief descriptive alternative. An example path is illustrative and not a claim that the asset is bundled.

```
:::record attachment #example-planner-sticker-and-deco-culture
method: planner-sticker-and-deco-culture@1
path: assets/rain-sticker.png
media-type: image/png
alt: A small rain cloud
role: decorative
:::
```

13.32 Focus rounds and interruption logs

Profile: `pomodoro-log@1` | Record type: `session`

Plan a bounded work round, record its actual outcome and park interruptions. Keep the task and the round as separate concepts.

Writing invitation: What will this round focus on, and what interrupted it?

Complete structured record: `focus`, `status`. Optional fields: `planned-duration`, `duration`, `duration-unit`, `start`, `end`, `interruption`.

status is planned, ongoing, completed or interrupted. interruption is repeatable. A completed round does not complete the linked task. Alternate round lengths are labeled adaptations, and the journal does not require a running timer.

```
:::record session #example-pomodoro-log
method: pomodoro-log@1
focus: Read the connector datasheet
status: completed
planned-duration: 25 min
duration: 25
duration-unit: min
:::
```

13.33 Life review and written dialogue

Profile: `progoff-intensive-journal@1` | Record type: `reflection`

Explore a period of life through turning points, dialogue and linked sections. A new observation can lead to another section without forcing a single chronological narrative.

Writing invitation: What period am I in, and what other part of my life does this connect to?

Complete structured record: `section`, `body`. Optional fields: `relation`, `voice`, `period-start`, `period-end`.

This is a general life-review adaptation, not a replacement for the official Intensive Journal workbook or workshop sequence. The writer controls pace and interpretation. Dialogue voices can use explicit voice cues in ordinary writing.

```
:::record reflection #example-progoff-intensive-journal
method: progoff-intensive-journal@1
section: Turning points
body: The move changed which routines felt like mine.
:::
```

13.34 Prompt-based writing

Profile: `prompt-journaling@1` | Record type: `response`

Start from a chosen question and keep the question with the answer. A prompt may be answered, adapted, postponed or skipped.

Writing invitation: What would help me notice something specific today?

Complete structured record: `question`, `status`. Optional fields: `answer`, `prompt`, `prompt-version`, `source`.

status is answered, partial, skipped or postponed. answer is required only when answered. A linked prompt is optional when question is stored locally; changing a prompt template must not change an existing response.

```
:::record response #example-prompt-journaling
method: prompt-journaling@1
question: What made the room feel welcoming?
status: answered
answer: A lamp instead of the overhead light.
:::
```

13.35 Paper scans and transcription

Profile: `rocketbook-and-scan-systems@1` | Record type: `attachment`

Identify a physical page and keep its scan, optional transcription and routing choices linked. The paper original remains a legitimate source.

Writing invitation: Which page is this, where is the original, and has the text been checked?

Complete structured record: `path`, `media-type`, `alt`. Optional fields: `page-label`, `physical-location`, `ocr-status`, `captured-at`, `sha256`.

ocr-status is none, unreviewed or verified. Brand routing symbols and encoded dot patterns are not assumed interoperable. A printed page ID should encode a local opaque identifier, not private journal content.

```
:::record attachment #example-rocketbook-and-scan-systems
method: rocketbook-and-scan-systems@1
path: assets/page-018.jpg
media-type: image/jpeg
alt: Handwritten notes and a cable sketch
page-label: 18
ocr-status: unreviewed
:::
```

13.36 Personal support and safety-plan records

Profile: `safety-planning@1` | Record type: `collection`

Represent an existing, author-chosen support plan as a stable, accessible section. Preserve its intended order, review date and contact details without automatic crisis inference.

Writing invitation: What support plan have I chosen, and where can I find the current copy?

Complete structured record: `title`, `kind`. Optional fields: `warning-sign`, `coping`, `contact`, `environment`, `reviewed-on`, `member`.

warning-sign, coping, contact, environment and member are repeatable. This data profile is not a complete clinical plan template or emergency service. Actual contacts and steps come from the author's existing plan; no alerts or sharing occur just because the record exists.

```
:::record collection #example-safety-planning
method: safety-planning@1
title: My support plan
kind: support-plan
reviewed-on: 2026-10-25
coping: Sit in the familiar quiet room
:::
```

13.37 Curiosity and growth reflections

Profile: `shadow-growth-journal@1` | Record type: `reflection`

Notice a reaction, possible meaning and a compassionate next step. Interpretations remain tentative and can be revised without overwriting the original feeling.

Writing invitation: What did I notice in my reaction, and what else could be true?

Complete structured record: `reaction`, `body`. Optional fields: `hypothesis`, `strength`, `boundary`, `closing`.

Do not infer hidden motives, recovered memories or personality diagnoses. A prompt can be skipped; the software must not insist that mistreatment is a projection or the writer's fault.

```
:::record reflection #example-shadow-growth-journal
method: shadow-growth-journal@1
reaction: Envy when a friend finished a project
body: I may want more protected making time.
strength: I care about creating things.
:::
```

13.38 Spoons and personal energy accounting

Profile: `spoon-theory-energy-accounting@1` | Record type: `observation`

Describe felt capacity, estimated costs and helpful recovery in a personally meaningful unit. The account is a metaphor and a planning aid, not a physiological measurement.

Writing invitation: What capacity do I feel I have, and where would I like to spend it?

Complete structured record: `unit`, `status`. Optional fields: `available`, `cost`, `recovery`, `balance`, `at`, `body`.

Free-text costs are valid reflection. Arithmetic views require numeric cost/recovery fields in separate measurement records with explicit scope. Never compare spoon counts between people or automatically increase activity targets.

```
:::record observation #example-spoon-theory-energy-accounting
method: spoon-theory-energy-accounting@1
unit: personal-spoons
status: estimated
available: 4
cost: Shopping felt like 2
body: Rest mattered more than finishing the list.
:::
```

13.39 Stims and regulation supports

Profile: `stimming-and-self-regulation@1` | Record type: `observation`

Keep a personal menu of helpful sensations and tools, plus optional notes about when they helped. The purpose is access and self-knowledge.

Writing invitation: What felt regulating, and would I like it available again?

Complete structured record: `support`. Optional fields: `context`, `effect`, `preference`, `body`.

support is repeatable. The profile has no reduction goal, compliance metric or required frequency count. The writer may prefer a static support menu with no daily monitoring at all.

```
:::record observation #example-stimming-and-self-regulation
method: stimming-and-self-regulation@1
support: Smooth stone in a pocket
context: Waiting in a noisy room
effect: Easier to stay present
:::
```

13.40 Stoic-inspired morning and evening review

Profile: `stoic-journaling@1` | Record type: `reflection`

Distinguish an event from a judgment and choose a response within personal influence. Morning intentions and evening reflection can remain simple prose.

Writing invitation: What can I influence, and what response fits my values?

Complete structured record: `event`, `response`. Optional fields: `judgment`, `within-influence`, `outside-influence`, `phase`, `learning`.

Acknowledging limited control does not excuse harm or demand emotional suppression. phase is morning or evening when supplied. Quotes should carry attribution and stay distinct from the writer's own reflections.

```
:::record reflection #example-stoic-journaling
method: stoic-journaling@1
event: The bus was late.
response: Leave a written note about the missed connection.
within-influence: How I report it
:::
```

13.41 Study, Cornell notes and retrieval practice

Profile: `study-and-note-taking@1` | Record type: `reflection`

Keep notes alongside questions that can later cue recall, plus a short summary in the writer's own words. Practice results can be logged separately.

Writing invitation: What question would test whether I understand this?

Complete structured record: `topic`, `note`, `cue`, `summary`. Optional fields: `source`, `review-on`, `recall-result`.

note and cue are repeatable. Pair cues and notes explicitly by separate records when order alone is insufficient. Review dates are suggestions, not a hidden executable spaced-repetition algorithm.

```
:::record reflection #example-study-and-note-taking
method: study-and-note-taking@1
topic: Pull-up resistors
note: They establish a default logic level.
cue: What happens when the input is otherwise floating?
summary: The resistor gives an unconnected input a defined default.
:::
```

13.42 Symptoms, sleep and care observations

Profile: `symptom-tracking@1` | Record type: `measurement`

Record an observation, unit/scale and time so it can be remembered accurately. Medication events, symptom ratings and possible contextual factors remain distinct data.

Writing invitation: What did I observe, when, and on which scale?

Complete structured record: `metric`, `status`. Optional fields: `value`, `unit`, `scale`, `at`, `recall`, `body`.

value is required for observed/estimated status and omitted for missing states. A perceived association is not causation. Standard clinical instruments require exact wording, version and permissions before an application claims compatibility.

```
:::record measurement #example-symptom-tracking
method: symptom-tracking@1
metric: sleep-duration
status: estimated
value: 7.5
unit: h
body: Approximate time asleep, not time in bed.
:::
```

13.43 Themes and directional review

Profile: `theme-system-journal@1` | Record type: `reflection`

Choose a direction for a period and notice actions that moved toward it. A theme can evolve and need not become a numerical success test.

Writing invitation: Where did I make a little more room for my theme?

Complete structured record: `theme`, `observation`. Optional fields: `period-start`, `period-end`, `action`, `alignment`, `scale`, `anti-theme`.

action is repeatable. Numeric alignment is optional and requires a scale. Partial participation remains meaningful; a theme is not automatically a pass/fail goal. Anti-theme is an optional personal label.

```
:::record reflection #example-theme-system-journal
method: theme-system-journal@1
theme: More room to breathe
observation: Left a gap between errands.
action: Paused before adding another task
:::
```

13.44 Time blocking and replanning

Profile: `time-block-planner@1` | Record type: `event`

Assign a block to an activity, revise the plan when circumstances change, and retain the distinction between intended and actual time.

Writing invitation: What is this block for, and does the remaining plan still fit?

Complete structured record: `title`, `start`, `end`, `mode`. Optional fields: `relation`, `revision`, `reason`.

mode is planned or actual. Replanned records use revises links rather than pretending the original plan happened. Overlaps are allowed with a visible warning, since real activities can overlap.

```
:::record event #example-time-block-planner
method: time-block-planner@1
title: Project reading
start: 2026-10-25T14:00:00-07:00
end: 2026-10-25T14:30:00-07:00
mode: planned
:::
```

13.45 Actual-time logs and day reconstruction

Profile: `time-logging-lifelog@1` | Record type: `session`

Describe actual activity intervals, with optional category and feeling. Retrospective reconstruction is labeled rather than presented as an exact live log.

Writing invitation: What was I actually doing, and how certain am I about the time?

Complete structured record: `activity`, `start`, `end`. Optional fields: `category`, `recall`, `confidence`, `feeling`.

The example is a completed interval; ongoing capture may omit end and use status ongoing. Weekly totals must state how overlaps, gaps and multi-category sessions are handled. Unlogged time is unknown, not wasted.

```
:::record session #example-time-logging-lifelog
method: time-logging-lifelog@1
activity: Reading on the bus
start: 2026-10-25T15:10:00-07:00
end: 2026-10-25T15:35:00-07:00
recall: retrospective
:::
```

13.46 Modular notebooks and travel-style inserts

Profile: `travelers-notebook@1` | Record type: `collection`

Group writing into replaceable inserts, topics or journeys, keeping an index and optional physical location. One .jot can hold several inserts conceptually.

Writing invitation: Which insert or journey does this belong to?

Complete structured record: `title`, `kind`. Optional fields: `member`, `physical-location`, `period-start`, `period-end`.

member is repeatable. Archiving an insert preserves cross-links and physical page locators. A digital collection does not claim exact physical compatibility with a commercial cover or refill.

```
:::record collection #example-travelers-notebook
method: travelers-notebook@1
title: Autumn carry notebook
kind: insert
physical-location: Small black notebook, second insert
:::
```

13.47 Free diary and stream of consciousness

Profile: `free-diary@1` | Record type: `reflection`

Keep narrative, fragments, contradictions and ordinary details without an imposed method.

Writing invitation: What do I want to write?

Complete structured record: `body`. Optional fields: `date`, `title`.

No date, length, mood score or conclusion is required for Core prose. This profile exists only for applications that want an explicit free-writing label.

```
:::record reflection #example-free-diary
method: free-diary@1
body: I have no plan for this paragraph. That is fine.
:::
```

13.48 Shared and dialogue journals

Profile: `shared-journal@1` | Record type: `response`

Keep each person's words and replies attributable, with a deliberate share scope.

Writing invitation: What do I want to share, and what response would I welcome?

Complete structured record: `author`, `body`. Optional fields: `relation`, `audience`, `consent`.

A response can link to an earlier record using responds-to. Sharing this file reveals its bytes; private writing requires separate access-controlled storage. No automatic invitation or message is implied.

```
:::record response #example-shared-journal
method: shared-journal@1
author: writer-b
body: I liked reading your description of the rain.
:::
```

13.49 Travel, nature and field observations

Profile: `travel-field-log@1` | Record type: `observation`

Capture what was seen, where and under what conditions, separating observations from identification guesses.

Writing invitation: What did I actually notice here?

Complete structured record: `observation`. Optional fields: `place`, `at`, `weather`, `hypothesis`, `asset`.

Precise coordinates are optional and excluded from sharing by default. Asset paths need real attachments before packaging. Uncertain identifications remain uncertain.

```
:::record observation #example-travel-field-log
method: travel-field-log@1
observation: Three small birds on the reed stems.
place: Riverside path
hypothesis: Species uncertain
:::
```

13.50 Reading and media journals

Profile: `reading-media-journal@1` | Record type: `reflection`

Connect a book, film, game or other work to a response, keeping spoilers and quotations attributable.

Writing invitation: What stayed with me from this work?

Complete structured record: `work`, `body`. Optional fields: `creator`, `locator`, `quotation`, `spoiler`, `source`, `series`.

spoiler is true or false. Repeated source and locator fields are allowed; `series` names the series the work belongs to (§4.2.9). A reaction is distinct from a factual summary, and reading progress is optional.

```
:::record reflection #example-reading-media-journal
method: reading-media-journal@1
work: A novel I am reading
body: The quiet scene mattered more than the ending.
spoiler: false
:::
```

13.51 Decision journals

Profile: `decision-journal@1` | Record type: `reflection`

Preserve the information available at decision time and compare it with a later outcome without rewriting the forecast.

Writing invitation: What do I expect, why, and when would I reconsider?

Complete structured record: `decision`, `reason`. Optional fields: `option`, `prediction`, `probability`, `review-on`, `outcome`.

option is repeatable. probability, when supplied, is numeric 0 through 1 and belongs to a specific prediction. Review records link back to the original; an outcome must not replace the original expectation.

```
:::record reflection #example-decision-journal
method: decision-journal@1
decision: Test the smaller enclosure first
reason: It will reveal the fit problem sooner.
prediction: The connector may need more clearance.
:::
```

13.52 Project and lab notebooks

Profile: `project-lab-log@1` | Record type: `observation`

Record a question, procedure, actual observation and next experiment, preserving revisions and source artifacts.

Writing invitation: What did I try, what happened, and what should I test next?

Complete structured record: `question`, `observation`. Optional fields: `procedure`, `configuration`, `asset`, `next-step`.

procedure and asset are repeatable. Configuration and units must be explicit when comparisons depend on them. A personal project log does not automatically satisfy regulated lab-record requirements.

```
:::record observation #example-project-lab-log
method: project-lab-log@1
question: Does the new cable remove the flicker?
observation: Flicker remained at the same brightness.
next-step: Inspect the supply connection.
:::
```

13.53 Food, cooking and meal memories

Profile: `food-cooking-journal@1` | Record type: `reflection`

Keep meals, recipe changes and sensory preferences as practical or autobiographical writing.

Writing invitation: What tasted good, and what would I change next time?

Complete structured record: `body`. Optional fields: `meal`, `recipe`, `ingredient`, `substitution`, `context`.

ingredient and substitution are repeatable. Nutrient estimates, if added, require separate source-labeled measurements. This profile does not require calorie tracking or label food as morally good/bad.

```
:::record reflection #example-food-cooking-journal
method: food-cooking-journal@1
body: The lentil soup was better the next day.
substitution: Used extra olives.
:::
```

13.54 Spiritual, ritual and meaning journals

Profile: `spiritual-reflection@1` | Record type: `reflection`

Allow prayer, secular ritual, values, questions and meaning-making in the writer's own language.

Writing invitation: What felt meaningful or worth sitting with?

Complete structured record: `body`. Optional fields: `tradition`, `ritual`, `question`, `source`.

No belief system, supernatural claim or preferred interpretation is imposed. Quoted sacred or literary text carries attribution; the writer may leave questions unresolved.

```
:::record reflection #example-spiritual-reflection
method: spiritual-reflection@1
body: Lighting the candle gave the evening a boundary.
ritual: A quiet minute before writing
:::
```

13.55 Memory, grief and life-story writing

Profile: `memory-life-story@1` | Record type: `reflection`

Keep a memory, the feelings around it and uncertainty about details. Several accounts may coexist.

Writing invitation: What do I remember, and what am I unsure about?

Complete structured record: `body`. Optional fields: `person`, `period`, `confidence`, `asset`.

Remembering is not verification of an event. No generated detail should be inserted as remembered fact. Resurfacing and sharing memories are explicit choices.

```
:::record reflection #example-memory-life-story
method: memory-life-story@1
body: I remember the music more clearly than the room.
confidence: Details may have changed in memory.
:::
```

13.56 Voice practice and voice training

Profile: `voice-practice@1` | Record type: `session`

Log voice practice on your own terms: what you practised, for how long, how your voice felt and, if you like, a median pitch or the name of a clip. Useful for gender-affirming voice training, singing or speech practice. The writer sets the goals.

Writing invitation: What did I practise, how did my voice feel, and did it feel like me?

Complete structured record: `minutes`, `exercise`. Optional fields: `pitch-hz`, `resonance`, `strain`, `felt-like-me`, `recording`, `goal`, `at`.

`minutes`, `pitch-hz`, `resonance` and `felt-like-me` are structured numeric values (§5.2) so a chart (§5.2.1) can draw them; `pitch-hz` is a median fundamental frequency in hertz from whichever tool the writer uses, and is optional. `resonance` is an ordinal scale whose anchors the writer chooses (for example "darker" and "brighter"); `felt-like-me` is 1 to 5. A writer who wants fixed axes declares scale records and writes those ratings as measurements naming them. `strain` is `none`, `some` or `a lot`. `recording` names a clip attached to a `.jotz` (section 2.0) or kept elsewhere; a reader MUST NOT fetch or play it without the writer's action. The profile is not medical advice, not an assessment instrument and not a substitute for a voice clinician. Implementations MUST NOT supply a gender target, a "passing" score, a comparison with other people's voices or a requirement to record dysphoria, and MUST NOT infer gender, health or progress from the numbers. Voice clips and pitch figures can identify a person: exports and shared copies SHOULD leave them out unless selected. The voice log in the gender-euphoria-and-hrt-log guide links here.

```
:::record session #example-voice-practice
method: voice-practice@1
minutes: 12
exercise: straw phonation, then reading aloud
pitch-hz: 182
resonance: 3
strain: none
felt-like-me: 4
recording: voice-2026-10-27.m4a
body: The warm-up made the reading easier.
:::
```

Scales that make the two ratings chart on their own axes:

```
:::record scale #felt-like-me-scale
name: felt like me
kind: ordinal
minimum: 1
maximum: 5
anchor-low: Did not feel like me
anchor-high: Felt like me
:::

:::record measurement #felt-like-me-20261027
method: voice-practice@1
metric: felt-like-me
value: 4
scale: #felt-like-me-scale
status: observed
:::
```

13.57 Task initiation and transitions (AuDHD)

Profile: `task-initiation-and-transitions@1` | Record type: `task`

Get from one activity to the next with a tiny first step, a cue that marks the switch, an optional body double, and a time estimate set beside what actually happened. Written for AuDHD writers (autistic and ADHD) who find both starting and stopping hard.

Writing invitation: What am I leaving, what is the next tiny step, and what will tell me it is time to switch?

Complete structured record: `next-step`. Optional fields: `from`, `cue`, `warning`, `estimate-min`, `actual-min`, `body-double`, `started`, `at`.

`started` is `yes`, `partly`, `not yet` or `chose not to`; none of them is a failure state. `estimate-min` and `actual-min` are minutes as structured numeric values, kept so the writer can learn their own time, never to grade it; implementations MUST NOT compute a punctuality or productivity score from them. `warning` is how long before the switch a reminder helps (free text). A record may describe a transition with no task finished. Complements `adhd-journaling@1` (daily planning) and `interstitial-journaling@1` (notes between tasks) rather than replacing them. Diagnosis is neither required nor recorded.

```
:::record task #example-task-initiation-and-transitions
method: task-initiation-and-transitions@1
from: Drawing
next-step: Stand up and fill the water glass
cue: Timer song ends
warning: 10 min
estimate-min: 5
actual-min: 15
body-double: Video call with Sam
started: yes
:::
```

13.58 Low-demand check-in (AuDHD)

Profile: `low-demand-check-in@1` | Record type: `observation`

A check-in that asks almost nothing: capacity, sensory load and energy as single numbers, one word for what would help, and skipped as a complete answer.

Writing invitation: Only if I want to: how much capacity, how loud is the world, how much energy?

Complete structured record: `status`. Optional fields: `capacity`, `sensory-load`, `check-in-energy`, `need`, `scale`, `at`.

`status` is `observed`, `estimated` or `skipped`. A record with `status: skipped` and no numbers is complete and MUST NOT be shown or counted as a low value (§5.2). Implementations MUST NOT add streaks, missed-day totals, catch-up prompts or automatic backfill for this profile; a day with no check-in is drawn as a gap. Each number is on the writer's own scale; `scale` names one scale record that applies to all three numbers in the record and fixes their chart axes. The energy number is `check-in-energy`, not `energy`, so it never merges with the differently scaled energy of `symptom-tracking@1` or `spoon-theory-energy-accounting@1` (§5.2). A shared scale's anchors SHOULD be neutral ("low", "high"), because a high capacity and a high sensory load mean different things. Detailed sensory logs belong to `autistic-sensory-journaling@1`.

```
:::record observation #example-low-demand-check-in
method: low-demand-check-in@1
status: observed
capacity: 2
sensory-load: 4
check-in-energy: 2
need: quiet
:::
```

13.59 Access needs and scripts (AuDHD)

Profile: `access-needs-and-scripts@1` | Record type: `collection`

Keep a personal list of what helps, where, and the exact words to ask for it, so a writer does not have to improvise self-advocacy when overloaded.

Writing invitation: What helps me, and how would I like to ask for it?

Complete structured record: `need`. Optional fields: `helps`, `context`, `script`, `share-with`.

`need` and `helps` are repeatable. No diagnosis is required, recorded or implied, and the list is not a medical or legal accommodation document. `share-with` is the writer's own note about who may see an entry, not an access control; exports SHOULD include only the entries selected for a reader. A static support menu lives in `stimming-and-self-regulation@1`.

```
:::record collection #example-access-needs-and-scripts
method: access-needs-and-scripts@1
need: Written instructions
helps: A short list I can reread
context: Work meetings
script: Could you send that as a message so I can come back to it?
:::
```

13.60 Drawing practice sessions

Profile: `drawing-practice@1` | Record type: `session`

Log one drawing practice session: the subject, the medium, the skill in focus (gesture, value, perspective…), what worked and what to try next. Progress is seen by comparing drawings of the same focus, not by streaks or scores.

Writing invitation: What did I draw, what was I practising, what worked, and what will I try next time?

Complete structured record: `subject`, `medium`, `focus`. Optional fields: `minutes`, `reference`, `credit`, `what-worked`, `next-try`, `asset`, `alt`.

focus, reference, credit, asset and alt are repeatable; alt pairs with asset in order. minutes is a whole number of minutes actually spent, never required and never inferred from a timer. reference may be a description, an assets/ path or a URL; a reference made by someone else carries credit (creator, source, licence). No streak, score or grade fields: a day without practice records nothing. An attached image is optional and needs alt text; the record does not claim to reproduce the drawing.

```
:::record session #example-drawing-practice
method: drawing-practice@1
subject: Hands holding a mug
medium: Graphite, 2B
focus: gesture
minutes: 20
reference: My own left hand, in a mirror
what-worked: Long lines for the fingers kept them in proportion.
next-try: Draw the mug first, then wrap the hand around it.
:::
```

13.61 Design iterations and critique

Profile: `design-iteration-and-critique@1` | Record type: `review`

Record one version of a design or art project: the decision and its rationale, the critique it received and what changes next. Each version links to the one it revises, so the history reads in order without rewriting earlier thinking.

Writing invitation: What changed in this version and why? What did the critique say, and what will change next?

Complete structured record: `project`, `iteration`, `decision`, `rationale`. Optional fields: `critique`, `critique-from`, `change-next`, `asset`, `alt`.

critique, critique-from, change-next, asset and alt are repeatable; critique-from pairs with critique and alt with asset, in order. critique-from names a role (peer designer, client, usability participant), not a person, unless that person agreed to be named. Link to the previous version with relation: revises #id (section 6.2), so give each iteration a record ID; an unresolved link stays visible and never blocks writing. A later version never edits an earlier decision. Screens need alt text saying what changed, and annotations must not rely on colour alone (section 8.1).

```
:::record review #example-design-iteration-and-critique
method: design-iteration-and-critique@1
project: Library app sign-up
iteration: v3
decision: Move the password rules above the field.
rationale: Two of five testers missed the rules when they appeared only after an error.
critique: The rules read as a wall of text.
critique-from: Peer designer
change-next: Group the rules as a short checklist.
relation: revises #signup-v2
:::
```

13.62 Moodboards and colour palettes

Profile: `moodboard-and-palette@1` | Record type: `palette`

Gather a theme, mood words, credited references and a named colour palette before making something. Every colour has a hex value and a name, so the palette is readable without seeing the swatches.

Writing invitation: What is this board for, which words describe the feel, and which colours and references carry it?

Complete structured record: `theme`, `color`. Optional fields: `mood-word`, `asset`, `alt`, `credit`.

color is repeatable, one per line, written #RRGGBB name: # and six hex digits, a space, then a plain name; a colour without a name is incomplete. Renderers may draw a swatch beside each name, but colour is never the only carrier of meaning (section 8.1): write any role (background, accent, warning) in the name. mood-word, asset, alt and credit are repeatable; alt and credit pair with asset in order, and attachments follow section 8.3. A reference made by someone else carries credit (creator, source, licence). Hex values are screen colours, not print specifications.

```
:::record palette #example-moodboard-and-palette
method: moodboard-and-palette@1
theme: Rainy bookshop poster
mood-word: quiet
mood-word: warm
color: #2E3A46 slate night (background)
color: #C8A165 lamplight (accent)
color: #E9E2D3 paper (text)
:::
```

14. Implementation sequence and source register

14.1 A small, useful first implementation

Implement a writing-first reader and editor before structured forms: paragraphs, headings, date sections, scene labels, rapid logs, annotations, comments and lossless save. It must open both a one-day file and a month-long file, preserve unknown syntax and permit writing without a template. A line-by-line parser is a suitable implementation approach; styling and pagination can remain downstream, as in the separation described in the linked Fountain implementation README.

Next add optional record parsing, stable references, profile validation and portable attachment handling. Then add readable screen/PDF projections with privacy selection. Profile forms and queries come after the underlying text works reliably. Scan recognition is an optional import capability, not a prerequisite for writing a valid journal.

An implementation should expose explicit operations: open text, edit text, validate, preview, export selection, package assets and reconcile a conflict. Parsing must not send notifications, publish entries, alter the filesystem outside the chosen workspace, or run journal content. A JavaScript parser prototype and a browser workspace now accompany this specification; full profile validation and runtime conformance are not claimed. The included fixtures and schema are implementation aids.

14.2 Writing-first quick reference

Need	Write
Ordinary journaling	Ordinary paragraphs, fragments or lists
A new day inside the current file	`### 2026-10-25
A topic or method section	`#### Gratitude` or any heading you choose
A moment/place cue	`.0830 - KITCHEN`
A small task	`[ ] Photograph the label`
A completed task	`[x] Photograph the label`
An event / note / question / idea	`o` / `-` / `?` / `!` followed by writing
A short summary	`= A small repair and a quiet evening.`
A sidebar note	`[[Remember to bring the spare cable]]`
An explicit inner voice	`::voice ME`
Literal syntax	Backslash before a leading marker, or a code fence
Reliable queryable properties	Optional `:::record ...` block
A month of writing	Keep writing under new date headings in the same file

14.3 Design decisions and compatibility boundaries

Jot keeps Fountain's readability goal and familiar sections, synopses, forced scenes, emphasis, marginalia and page breaks. It replaces screenplay-specific assumptions with journal date sections, rapid capture and optional records. Fountain compatibility is not

claimed: dot-prefixed scenes, dialogue inference, title metadata and underline behavior must be handled by an explicit converter. The shared inspiration does not require inheriting a screenplay's layout or automatic uppercase character recognition.

The v1 metadata delimiters, optional record grammar and explicit voice cue are proposed decisions that make parsing deterministic. They can be revised before a stable v1 release. A stable release should freeze the grammar, publish an executable parser test suite, review all profile contracts, implement at least two independent readers, and document any interoperability issues discovered. User writing and existing original-outline sources should remain recoverable throughout that process.

14.4 Sources and scope of review

The source register records the original library snapshot, subsequent guide work and review scope. The methods catalogue maps all 62 profiles to their complete guides and evidence. These sources distinguish method descriptions, research findings and original Jot adaptations; profile fields are data-design choices, not clinical recommendations. The original coverage review did not independently audit every guide's bibliography.

Jot draws on Fountain's readable plain-text approach; it is a separate journaling format and claims no Fountain compatibility. See the source register for provenance and the baseline specification.

15. JavaScript, diagrams, handwriting and the project roadmap

15.1 Separate the language, workspace and scanner

Jot is the journal language and file. The JavaScript parser consumes writing; a web editor provides capture and presentation; a scanner processes page images. These components can interoperate without requiring the scanner to recognize handwriting or forcing all users into one app. The accompanying parser prototype runs as an ES module in Node and browsers. It uses vendored Marked 17.0.5 for GFM Markdown and adds source-preserving journal blocks. Prototype version 0.1 is a documented subset, not a complete implementation of this draft's semantic validators. Its README and tests define its current behavior.

15.2 Diagram and chart blocks

A fenced block labeled `mermaid`, `dot`/`graphviz`, `plantuml`/`puml`, `d2`, `vega`, or `vega-lite`/`vegalite` carries diagram source. A parser identifies the engine and retains the original code block, source language and payload. Vega/Vega-Lite configuration must be a JSON object for

structured interpretation; invalid input is preserved with a diagnostic. Other diagram engines can register language labels. Unknown engines remain ordinary code, making journals readable without plugins.

Recognition, syntax validation and rendering are distinct capabilities. Render adapters MUST declare the engine/version, limit resource usage, prevent unapproved remote loads and sanitize generated SVG/HTML. A remote PlantUML or other rendering service must not receive private journal text without an explicit user-selected action. Rendered output SHOULD carry a text alternative and link back to its source. The working prototype recognizes these blocks; their rendering engines are not installed yet.

15.3 Handwriting as original journal content

Handwriting does not require transcription to belong to a journal. Preserve editable strokes, a page-image fallback, page geometry and the author's intended ordering alongside typed writing. A `jot-ink` version 1 JSON asset has `format`, `version`, `page: {widthMm,heightMm}`, `coordinates: normalized`, and an ordered `strokes` list. Each stroke has `tool` (pen or eraser), `color` (#RRGGBB), `widthMm` and `points`, with point x/y/pressure values from 0 through 1. Eraser operations act on ink, not the page grid. Unknown or malformed stroke files must not execute code; import validates ranges and resource limits.

Link the stroke asset and PNG fallback through attachment records inside the same `.jotz` package. A later transcription is a separate derived record with review status; it MUST NOT replace the strokes or photograph. Pressure is optional input, not a biometric identity feature. Editors SHOULD provide undo/redo, adjustable pen width, a finger/stylus mode, and non-handwriting alternatives. Stylus-only input is a basic palm-rejection aid, not a guarantee across hardware. The current canvas supports capture, stroke export/import and inserting both a PNG fallback and editable stroke data into the journal. The editor packages them together inside `.jotz`; full visual block editing and handwriting recognition remain on the roadmap.

15.4 Writing assistance is optional

Spellcheck and grammar/style assistance are separate from parsing validity. An intentional fragment, dialect, name, poem or repeated word is not a malformed journal. Writing suggestions MUST be optional, non-destructive, associated with source ranges and separated from syntax diagnostics. Users should be able to ignore them without changing the journal or losing save/export capability.

The web workspace uses browser-native spellcheck. The JavaScript prototype optionally accepts a supplied lowercase word Set and produces dictionary-miss hints; it does not embed a complete spelling dictionary. The initial grammar module checks a few English patterns such as repeated words and punctuation spacing, restricted to prose rather than code or

structured measurements. This is not a comprehensive grammar engine. Full grammar adapters and additional languages remain planned. External checking services require explicit opt-in before receiving journal content.

15.5 A maintained roadmap

The accompanying format ROADMAP.md and separate WRITER-ROADMAP.md distinguish specified, implemented, tested, partial and planned features. Format changes require corresponding Writer support in the same implementation change; the format roadmap owns the contract and the Writer roadmap owns web, desktop/mobile and integration delivery. Every substantial change should update the roadmap, API/version notes, acceptance criteria and known limitations together. Priorities are: reliable writing and portable export; real-device scanner/handwriting checks; profile/link validation; attachment bundles and recovery; local rendering and recognition adapters; then a stable interoperability release. No feature is reported complete merely because its profile or syntax appears in this specification.